

Knowledge Graph Mastery

For AI Developers

From Fundamentals to GraphRAG

Neo4j • Cypher • LangChain • LangGraph • CrewAI • Google ADK • RAGAS

Neo4j

Cypher

LangChain

LangGraph

CrewAI

Google ADK

RAGAS

Agentic AI Coding Fitness • Week 9 | AltoTech Global

agentic-coding-fitness.altotech.ai • AltoTech Global

Reference: graphacademy.neo4j.com/knowledge-graph-rag/

Prerequisites: Python 3.11+ • Docker • Neo4j 5.x • graphacademy.neo4j.com/knowledge-graph-rag/

PART 1: Knowledge Graph Fundamentals

Nodes, Edges, Properties — The Building Blocks of Connected Intelligence

1.1 What Is a Knowledge Graph?

A knowledge graph is a network of real-world entities (nodes), interconnected by typed, directed relationships (edges). Every node and every relationship can carry arbitrary key-value properties. Together, these three primitives — nodes, edges, properties — form a semantic network that machines can traverse, reason over, and augment with machine-learning embeddings.

In Neo4j syntax: a node with label `:Person`, a relationship `-[:KNOWS]->` pointing to another node, and properties like `{name: 'Alice', since: 2022}`. The combination creates a first-class data model for connected, heterogeneous, schema-flexible data.

Data Model Comparison

Feature	RDBMS	Document DB	Knowledge Graph
Data model	Tables + rows	JSON documents	Nodes + edges + properties
Joins/traversal	Expensive JOIN ops	Embedded or ref lookup	Index-free adjacency — $O(1)$ hop
Schema flexibility	Rigid — ALTER TABLE	Flexible per-doc	Additive — new edge types never break schema
Multi-hop queries	Requires many JOINS	Complex \$lookup pipelines	Natural: MATCH (a)-[:R*1..5]->(b)
AI integration	SQL via ORM	Embedding stored in field	Native vector index + graph traversal combined
Best for	Transactional, tabular	Hierarchical, document-centric	Relationships, recommendations, RAG, memory

Why Graphs for AI Systems?

AI agents need to remember relationships, not just facts. A vector database tells you "these chunks are semantically similar." A knowledge graph tells you why — because Device C-7 is installed in Room 301 which is on Floor 3 which is managed by Somchai. That causal, relational context is what separates hallucinating AI from grounded AI.

Key advantages: (1) Connected data — multi-hop traversal in a single query. (2) No schema migrations — add a new relationship type without touching existing data. (3) Explainability — every answer traces back to a path in the graph. (4) Hybrid retrieval — combine vector similarity with graph-structure filters.

Woolridge (MAS, Ch.2) — Agent Belief Structures

An agent's beliefs are its internal model of the world. Knowledge graphs are the most natural structure for representing and querying those beliefs. A belief node is a labelled entity; a belief update is a MERGE or SET operation; belief revision is a graph query that identifies contradictions. Woolridge's BDI (Belief-Desire-Intention) architecture maps directly onto graph primitives: beliefs = nodes/edges, desires = goal nodes, intentions = active plan subgraphs.

1.2 Neo4j Architecture

Neo4j is a native graph database — it stores and processes graphs using a purpose-built engine, not an adapter on top of a relational or document store. The core innovation is index-free adjacency: each node stores a direct pointer to its adjacent nodes in memory, so a one-hop traversal is $O(1)$ regardless of graph size. This makes multi-hop queries — the foundation of GraphRAG — dramatically faster than any JOIN-based approach.

Neo4j supports full ACID transactions (Atomicity, Consistency, Isolation, Durability), making it suitable for production workloads where data integrity matters. The Bolt protocol (port 7687) is the binary wire protocol for drivers; the HTTP API (port 7474) exposes the browser console and REST endpoints.

Component	Role	Key Commands / Notes
Neo4j Core	ACID native graph database — stores nodes, edges, properties with index-free adjacency	CREATE, MATCH, MERGE, SET, DELETE, RETURN
APOC	Extended procedures library — 450+ functions for data transformation, format conversion, graph algorithms	apoc.load.json(), apoc.create.node(), apoc.periodic.iterate(), apoc.refactor.mergeNodes()
GDS (Graph Data Science)	ML algorithms on graphs — PageRank, community detection, node embeddings, path finding	gds.pageRank.stream(), gds.louvain.stream(), gds.fastRP.stream(), gds.node2vec.write()
Bloom	Visual graph exploration — point-and-click graph browser with search phrases	GUI tool — no Cypher required; ships with Neo4j Desktop
LLM Graph Builder	Auto-extract entities and relationships from PDFs, URLs, YouTube, text	Docker / Cloud — React UI + Python backend; uses OpenAI / Claude / Gemini

1.3 Cypher Query Language — Basics

Cypher is Neo4j's declarative, ASCII-art graph query language. It was designed to be readable by humans who can visualise graph patterns: nodes are parentheses (n), relationships are arrows -[:TYPE]->, and patterns compose naturally. Cypher is now an open standard (GQL) adopted by multiple graph databases.

The twelve basic patterns below cover 90% of day-to-day graph operations. Master these before moving to the intermediate and advanced sections.

1. Create a Node

```
CREATE (n:Person {name: 'Alice', age: 30})
RETURN n
```

Creates a node with label :Person and two properties. RETURN n shows the node.

2. Create a Relationship

```
MATCH (a:Person {name:'Alice'}),
      (b:Person {name:'Bob'})
CREATE (a)-[:KNOWS {since: 2020}]->(b)
```

First MATCH both nodes, then CREATE the typed relationship with a property.

3. MATCH + RETURN

```
MATCH (n:Person)
RETURN n.name AS name, n.age AS age
ORDER BY n.age DESC LIMIT 10
```

Basic read query — find all Person nodes, return sorted properties.

4. WHERE Filter

```
MATCH (n:Person)
WHERE n.age >= 25 AND n.name STARTS WITH 'A'
RETURN n.name, n.age
```

WHERE supports AND/OR, STARTS WITH, ENDS WITH, CONTAINS, regex, IN list.

5. MERGE (Upsert)

```
MERGE (n:Person {name: $name})
ON CREATE SET n.created = datetime(),
n.age = $age
ON MATCH SET n.last_seen = datetime()
RETURN n
```

MERGE = find or create. ON CREATE / ON MATCH let you handle both cases differently.

6. WITH + Aggregation

```
MATCH (p:Person)-[:KNOWS]->(friend)
WITH p, count(friend) AS friend_count
WHERE friend_count > 3
RETURN p.name, friend_count
ORDER BY friend_count DESC
```

WITH passes results forward and enables post-aggregation filtering with WHERE.

7. UNWIND (List Expansion)

```
UNWIND ['Alice','Bob','Carol'] AS name
MERGE (p:Person {name: name})
RETURN p
```

UNWIND expands a list into individual rows — great for bulk MERGE from Python lists.

8. DELETE / DETACH DELETE

```
MATCH (n:Person {name: 'Alice'})
DETACH DELETE n -- removes node AND all relationships
```

DELETE alone fails if the node has relationships. DETACH DELETE removes all edges first.

9. SET / REMOVE Properties

```
MATCH (n:Person {name: 'Alice'})
SET n.email = 'alice@example.com',
n.active = true
REMOVE n.temp_field
```

SET adds or updates properties. REMOVE deletes a property or label from a node.

10. OPTIONAL MATCH

```
MATCH (p:Person)
OPTIONAL MATCH (p)-[:MANAGES]->(team)
RETURN p.name, team.name AS team_or_null
```

Like SQL LEFT JOIN — returns null for team if no MANAGES relationship exists.

11. Named Relationship

```
MATCH (a:Person)-[r:KNOWS]->(b:Person)
WHERE r.since >= 2022
RETURN a.name, b.name, r.since
```

Naming the relationship [r] lets you filter and return relationship properties.

12. Counting + EXISTS

```
MATCH (p:Person)
WHERE EXISTS { (p)-[:KNOWS]->(:Person) }
```

```
RETURN p.name,  
size([(p)-[:KNOWS]->(c) | 1]) AS connections
```

EXISTS subquery checks for a pattern. `size([pattern|expr])` counts matches inline.

1.4 Cypher Tutorial — Intermediate Patterns

Intermediate Cypher covers multi-hop traversal, path algorithms, aggregation patterns, and subqueries. These patterns unlock real graph power: reasoning over connected data at variable depth.

1. Multi-Hop Path Matching

The `*1..3` syntax matches paths of 1 to 3 hops. This is the fundamental pattern for graph-distance queries — "find all friends of friends up to depth 3."

Multi-Hop Path Matching

```
MATCH path = (a:Person {name:'Alice'})-[:KNOWS*1..3]->(b)  
RETURN b.name AS friend, length(path) AS hops  
ORDER BY hops
```

2. Shortest Path

`shortestPath()` finds the minimum-hop path between two nodes using BFS. For weighted shortest paths, use GDS Dijkstra (Section 1.5).

Shortest Path (BFS)

```
MATCH (start:Room {id:'R101'}),  
      (end:Room {id:'R305'})  
MATCH p = shortestPath((start)-[*]-(end))  
RETURN p, length(p) AS distance
```

3. Variable-Length Relationships

Variable-length relationships (`[[:TYPE]*min..max]`) are essential for supply chain, org hierarchy, and network analysis. Always set a maximum depth to avoid full-graph traversals.

Variable-Length Relationships

```
MATCH (source)-[:SUPPLIES*1..5]->(target:Product)  
WHERE source.country = 'CN'  
RETURN source.name, target.name,  
length()-[:SUPPLIES*1..5]->() AS chain_length
```

4. COLLECT / List Comprehension

`collect()` aggregates values into a list. List comprehensions `[x IN list | expression]` transform lists inline — equivalent to Python list comprehensions.

COLLECT and List Comprehension

```
MATCH (dept:Department)-[:HAS]->(emp:Employee)  
RETURN dept.name,  
collect(emp.name) AS employees,  
[e IN collect(emp) | e.salary] AS salaries
```

5. CASE Expression

CASE works like SQL CASE WHEN — useful for computing derived categorical values from numeric properties at query time.

CASE Expression

```
MATCH (r:Room)  
RETURN r.id,  
CASE  
WHEN r.temp_c > 28 THEN 'HOT'  
WHEN r.temp_c < 18 THEN 'COLD'  
ELSE 'COMFORTABLE'
```

```
END AS comfort_status
```

6. FOREACH (Side Effects in Paths)

FOREACH executes write operations for each element in a list or path. It's used for marking all nodes along a path — e.g., flagging traversed nodes in a graph algorithm.

FOREACH — Side Effects on Paths

```
MATCH path = (a)-[:KNOWS*]->(b {name:'Bob'})
FOREACH (n IN nodes(path) |
  SET n.visited = true
)
```

7. Subquery with CALL {}

CALL {} subqueries let you write correlated subqueries — queries that depend on outer-scope variables. This is Neo4j's equivalent of SQL correlated subqueries, and is essential for complex aggregations.

CALL {} Subquery

```
MATCH (p:Person)
CALL {
  WITH p
  MATCH (p)-[:KNOWS]->(friend)
  RETURN count(friend) AS friend_count
}
RETURN p.name, friend_count
```

8. EXISTS Subquery

EXISTS {} subqueries check whether a pattern exists without returning data. Use them to filter nodes based on complex relationship conditions.

EXISTS Subquery

```
MATCH (p:Person)
WHERE EXISTS {
  MATCH (p)-[:MANAGES]->(t:Team)
  WHERE t.size > 5
}
RETURN p.name AS manager
```

9. Date/Time Functions

Neo4j has first-class temporal types: datetime(), date(), time(), and duration(). Use these for time-series IoT data, event graphs, and temporal reasoning.

Date/Time Filtering

```
MATCH (e:Event)
WHERE e.ts > datetime() - duration({days: 7})
AND e.ts.dayOfWeek IN [1, 2, 3, 4, 5]
RETURN e.type, e.ts.date AS date
```

10. Pattern Comprehension

Pattern comprehensions [(pattern) | expression] collect results of a pattern inline, without a separate MATCH clause. Ideal for building lists of related entities per node.

Pattern Comprehension

```
MATCH (p:Person)
RETURN p.name,
[(p)-[:KNOWS]->(f) WHERE f.age > 25 | f.name]
AS older_friends
```

Advanced Cypher covers APOC procedures, full-text and vector indexes, and Graph Data Science (GDS) algorithms. These patterns are the bridge between a data store and an intelligent, ML-augmented knowledge graph.

1. APOC Procedures

APOC (Awesome Procedures On Cypher) is the standard extension library for Neo4j. It provides over 450 procedures for loading data, graph refactoring, and batch processing.

APOC Procedures

```
-- Load JSON from URL
CALL apoc.load.json('https://api.example.com/data')
YIELD value
MERGE (n:Item {id: value.id})
SET n += value.properties

-- Graph refactoring
CALL apoc.refactor.mergeNodes(
  [node1, node2], {properties:'combine'}
)

-- Batch create (avoid OOM on large imports)
CALL apoc.periodic.iterate(
  "MATCH (n:Room) WHERE n.processed IS NULL RETURN n",
  "SET n.processed = true",
  {batchSize: 100}
)
```

2. Full-Text Index

Full-text indexes use Lucene under the hood. They enable BM25-ranked keyword search across multiple properties — essential for hybrid retrieval (keyword + vector + graph).

Full-Text Index (Lucene)

```
-- Create index
CREATE FULLTEXT INDEX room_search
FOR (r:Room) ON EACH [r.description, r.type]

-- Query with relevance score
CALL db.index.fulltext.queryNodes(
  'room_search', 'luxury suite oceanview'
) YIELD node, score
RETURN node.id, node.type, score
ORDER BY score DESC
```

3. Vector Index (AI Embeddings)

Neo4j native vector indexes (available since v5.11) store embedding vectors alongside graph nodes. This enables K-NN similarity search that can be immediately followed by graph traversal — the core of GraphRAG.

Vector Index — KNN Search

```
-- Create vector index (1536-dim for OpenAI ada-002)
CREATE VECTOR INDEX chunk_embeddings
FOR (c:Chunk) ON (c.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}}

-- K-NN similarity search
CALL db.index.vector.queryNodes(
  'chunk_embeddings', 5, $query_embedding
) YIELD node, score
RETURN node.text, score
```

4. GDS — PageRank

Graph Data Science PageRank identifies the most influential nodes in a graph. In an IoT graph, high-PageRank devices are the most critical to monitor; in a knowledge graph, they are the most connected entities.

GDS — PageRank

```
-- Step 1: Project a named in-memory graph
CALL gds.graph.project(
  'socialGraph',
  'Person',
  'KNOWS'
)
-- Step 2: Run PageRank
CALL gds.pageRank.stream('socialGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name,
score
ORDER BY score DESC LIMIT 10
```

5. GDS — Louvain Community Detection

Louvain detects clusters of tightly connected nodes — communities. In hotel IoT: rooms with shared HVAC zones form communities. In a knowledge graph: topic clusters emerge automatically.

GDS — Louvain Community Detection

```
CALL gds.louvain.stream('socialGraph')
YIELD nodeId, communityId
RETURN gds.util.asNode(nodeId).name,
communityId
ORDER BY communityId, name
```

6. GDS — Node Embeddings (FastRP)

FastRP (Fast Random Projection) generates graph-structural embeddings for nodes. Unlike text embeddings, FastRP captures graph topology — nodes that are structurally similar (same neighbourhood pattern) get similar embeddings, even if their labels differ.

GDS — FastRP Node Embeddings

```
CALL gds.fastRP.stream('socialGraph', {
  embeddingDimension: 128,
  iterationWeights: [0.0, 1.0, 1.0]
})
YIELD nodeId, embedding
RETURN gds.util.asNode(nodeId).name,
embedding
```

7. GDS — Shortest Path (Dijkstra)

Dijkstra's algorithm finds the minimum-cost path when relationships have weights. In a hotel building graph, edge weights are distances in meters; in a supply chain, they are transit times or costs.

GDS — Dijkstra Weighted Shortest Path

```
MATCH (source:Room {id:'R101'}),
(target:Room {id:'R305'})
CALL gds.shortestPath.dijkstra.stream(
  'roomGraph', {
    sourceNode: source,
    targetNode: target,
    relationshipWeightProperty: 'distance_m'
  })
YIELD path, totalCost
RETURN [n IN nodes(path) | n.id] AS route,
totalCost AS total_meters
```

8. Hybrid Vector + Graph Retrieval

This is the core GraphRAG pattern: use vector similarity to find relevant text chunks, then immediately traverse graph edges to expand context with related entities. The result is richer, more grounded context than pure vector search.

Hybrid Vector + Graph Retrieval (GraphRAG Core)

```
-- Find semantically similar chunks,
-- then expand to related entities via graph traversal
CALL db.index.vector.queryNodes(
  'chunk_embeddings', 3, $query_vec
) YIELD node AS chunk, score
MATCH (chunk)-[:MENTIONS]->(entity)
RETURN chunk.text, entity.name,
       entity.type, score
ORDER BY score DESC
```

1.6 Exercises — Cypher Practice

Complete these exercises in Neo4j Browser (localhost:7474) or Aura Console. Use the hotel IoT dataset loaded in the setup lab. Exercises progress from beginner to advanced.

Level	Exercise	Expected Solution (Cypher)
Beginner	Find all Room nodes. Return room id, floor, and type. Order by floor ascending.	<code>MATCH (r:Room) RETURN r.id, r.floor, r.type ORDER BY r.floor</code>
Beginner	Find all SensorReading nodes where temp_c > 30. Return room id and temperature.	<code>MATCH (r:Room)-[:HAS_READING]->(s:SensorReading) WHERE s.temp_c > 30 RETURN r.id, s.temp_c, s.ts</code>
Intermediate	Count how many devices are installed in each room. Return room id and device count, descending.	<code>MATCH (r:Room)-[:HAS_DEVICE]->(d:Device) WITH r, count(d) AS devices RETURN r.id, devices ORDER BY devices DESC</code>
Intermediate	Find rooms that triggered more than 2 alerts in the last 7 days. Return room id and alert count.	<code>MATCH (r:Room)-[:HAS_DEVICE]->(d:Device)-[:TRIGGERED]->(a:Alert) WHERE a.ts > datetime() - duration({days:7}) WITH r, count(a) AS alerts WHERE alerts > 2 RETURN r.id, alerts</code>
Advanced	Find all paths between Room R101 and any MaintenanceJob within 3 hops. Return the path length.	<code>MATCH path = (r:Room {id:'R101'})-[*1..3]->(m:MaintenanceJob) RETURN length(path) AS hops, [n IN nodes(path) labels(n)[0]] AS node_types</code>
Advanced	For each floor, compute average energy (kWh) in the last 24h. Rank floors by consumption descending.	<code>MATCH (r:Room)-[:HAS_READING]->(s:SensorReading) WHERE s.ts > datetime() - duration({hours:24}) WITH r.floor AS floor, avg(s.energy_kwh) AS avg_energy RETURN floor, round(avg_energy, 2) AS avg_kwh ORDER BY avg_kwh DESC</code>

Key Insight: Solutions shown above are reference answers — multiple correct Cypher formulations exist. Focus on: (1) correct pattern matching, (2) appropriate aggregation, (3) meaningful ORDER BY / LIMIT. Profile queries with EXPLAIN and PROFILE to understand execution plans.

1.7 Graph Data Modelling Deep Dive

Great knowledge graphs start with deliberate data modelling. This section walks through the complete modelling workflow for a hotel IoT system — the domain used throughout this course. The same workflow applies to any domain: healthcare records, supply chains, financial networks.

Step 1 — Identify the domain entities. List every "thing" your system needs to know about. For a hotel: rooms, devices, staff, guests, maintenance jobs, energy readings, alerts, suppliers. Each becomes a node label in Neo4j.

Step 2 — Identify relationships. For each pair of entities, ask: "Can A relate to B? If so, how?" Be specific. Not IS_RELATED_TO — instead: HAS_DEVICE, TRIGGERED, ASSIGNED_TO, CAUSED_BY, RESOLVED_BY. Each unique semantic relationship becomes a relationship type.

Step 3 — Identify properties. What attributes does each entity carry? Room: id, floor, type, capacity, rate_thb. Device: id, type, manufacturer, model, installed_at. Keep properties on the node where they describe the entity; put properties on the relationship where they describe the connection (since date, edge weight, severity).

Step 4 — Validate against queries. Write the 10 most important questions your system needs to answer, then check that your model supports all of them. If any query requires 8+ hops, rethink the model — add intermediate nodes, denormalise, or add helper edges.

Hotel IoT Domain Model — Node Reference

Node Label	Key Properties	Key Relationships	Cardinality
Room	id, floor, type, capacity, rate_thb, status	-[:HAS_DEVICE]->(Device), -[:HAS_READING]->(SensorReading)	1 Room to 0..N Devices, 0..M Readings
Device	id, type, model, manufacturer, installed_at, status	-[:TRIGGERED]->(Alert), -[:REPORTS_TO]->(Gateway)	1 Device to 0..N Alerts
SensorReading	ts, temp_c, humidity_pct, energy_kwh, occupancy	-[:SNAPSHOT_OF]->(Room)	M Readings per Room (time series)
Alert	id, type, severity, message, ts, resolved	-[:TRIGGERED_BY]->(Device), -[:ASSIGNED_TO]->(Staff)	1 Alert to 1 Device, 0..1 Staff assignment
Staff	id, name, role, shift, skills_list	-[:PERFORMED]->(MaintenanceJob), -[:ASSIGNED_TO]->(Alert)	1 Staff to 0..N Jobs
MaintenanceJob	id, type, description, started_at, completed_at, outcome	-[:RESOLVES]->(Alert), -[:PERFORMED_BY]->(Staff)	1 Job to 1..N Alerts (may resolve multiple)
Guest	id, name, check_in, check_out, preferences_list	-[:STAYED_IN]->(Room), -[:REPORTED]->(GuestIncident)	1 Guest to 1..N Room stays
Supplier	id, name, country, category, sla_days	-[:SUPPLIES]->(Room), -[:PROVIDES]->(Device)	1 Supplier to N Rooms or Devices

Graph Modelling Principle — Reify Relationships

"Reification" means turning a relationship into a node when the relationship itself has attributes or participates in other relationships. Example: A stay at a hotel is not just (Guest)-[:STAYED_IN]->(Room). It is a StayEvent node with check_in, check_out, room_rate properties, and its own relationships: -[:INVOICED_TO]->(Invoice), -[:INCLUDES]->(RoomService). Reify when: (1) the relationship has more than 2 properties, (2) other entities need to point to the relationship, (3) you need to query the relationship independently of its endpoints.

Cypher Schema Validation Queries

Use these Cypher queries to validate your domain model after loading data. Run them in Neo4j Browser to audit consistency and catch import errors.

Schema Validation Cypher Queries

```
-- Check for orphan nodes (nodes with no relationships)
MATCH (n)
WHERE NOT (n)--() AND NOT n:Chunk
RETURN labels(n), count(n) AS orphans

-- Check for missing required properties
MATCH (r:Room)
WHERE r.id IS NULL OR r.floor IS NULL
RETURN r, "MISSING_REQUIRED_PROPERTY" AS issue
```

```

-- Check for duplicate rooms (should be 0 if constraint exists)
MATCH (r:Room)
WITH r.id AS room_id, count(*) AS cnt
WHERE cnt > 1
RETURN room_id, cnt AS duplicates

-- Degree distribution (edges per node)
MATCH (n:Device)
RETURN size([(n)--() | 1]) AS degree,
count(n) AS node_count
ORDER BY degree

-- Schema overview (renders in Neo4j Browser)
CALL db.schema.visualization()

```

Key Insight: Run schema validation queries after every bulk import. Orphan nodes and NULL required properties are the two most common import errors. `CALL db.schema.visualization()` renders a visual schema graph in Neo4j Browser — use it to verify your model matches the design.

1.8 Temporal Knowledge Graphs

IoT and operational knowledge graphs are inherently temporal. Room temperature changes every minute; device status changes on alerts; staff assignments change per shift. Modelling time correctly is critical for any system that needs to answer questions like "what was the state of Room 301 at 14:00 on Tuesday?"

Neo4j supports two temporal modelling strategies: property-based (`valid_from` / `valid_to` on nodes) and event-sourcing (linking a timeline of event nodes). The choice depends on your query patterns and how important full historical audit is.

Temporal Modelling Patterns

```

-- Strategy 1: Property-based bi-temporal
MERGE (r:Room {id: 'R301'})
SET r.status = 'OCCUPIED',
r.valid_from = datetime('2024-01-15T14:00:00'),
r.valid_to = datetime('2024-01-16T12:00:00'),
r.recorded_at = datetime()

-- Query: room state at a specific point in time
MATCH (r:Room {id: 'R301'})
WHERE r.valid_from <= datetime('2024-01-15T16:00:00')
AND r.valid_to >= datetime('2024-01-15T16:00:00')
RETURN r.status

-- Strategy 2: Event-sourcing timeline
MATCH (r:Room {id: 'R301'})
CREATE (e:RoomEvent {
  ts: datetime(),
  event_type: 'STATUS_CHANGE',
  old_status: 'VACANT',
  new_status: 'OCCUPIED'
})
CREATE (r)-[:HAD_EVENT]->(e)

-- Query: room history last 24 hours
MATCH (r:Room {id: 'R301'})-[:HAD_EVENT]->(e)
WHERE e.ts > datetime() - duration({hours: 24})
RETURN e.event_type, e.ts, e.new_status
ORDER BY e.ts DESC

```

Strategy	Pros	Cons	Best For
Property-based (bi-temporal)	Simple queries; one node per entity; low storage overhead	Cannot represent full history; overwrites previous state on update	Current state queries; slowly-changing dimensions; simple CRUD
Event sourcing (timeline graph)	Full audit trail; point-in-time queries; append-only writes	More nodes and edges over time; queries need timestamp filtering	IoT time series; audit logs; temporal reasoning agents; compliance
Daily snapshot nodes	Efficient for aggregate queries over bounded time periods	Coarse granularity; gap between snapshots loses sub-period events	Daily energy reports; end-of-day room occupancy summaries

1.9 Multi-Label Nodes and Ontologies

Neo4j nodes can carry multiple labels, enabling type hierarchies and ontology-aligned modelling. A node labelled both `:Device` and `:HVACUnit` can be queried at either the general or specific level. This pattern is especially useful when integrating with external ontologies (schema.org, OWL, RDF-compatible schemas).

Use multi-label nodes when: (1) you have a type hierarchy where nodes are instances of multiple classes, (2) you want queries at different abstraction levels, (3) you are aligning to an external ontology. Avoid them for attribute-like classification — use a property instead.

Multi-Label Nodes — Type Hierarchy and Dynamic Labels

```
-- Multi-label node: both Device and HVACUnit and SmartDevice
CREATE (d:Device:HVACUnit:SmartDevice {
  id: 'HVAC-C7',
  model: 'Daikin VRV-IV',
  capacity_kw: 5.5
})

-- Query at broad level: all devices
MATCH (d:Device) RETURN count(d)

-- Query at specific level: only HVAC units
MATCH (d:HVACUnit) RETURN d.id, d.model

-- Intersection query: both SmartDevice AND HVACUnit
MATCH (d:SmartDevice:HVACUnit) RETURN d.id

-- Add a label dynamically (processing state pattern)
MATCH (d:Device {id: 'HVAC-C7'})
SET d:MaintenanceRequired

-- Remove a processing label
MATCH (d:Device {id: 'HVAC-C7'})
REMOVE d:MaintenanceRequired
```

Label Best Practices

USE labels for: (1) node type classification (`:Room`, `:Device`, `:Staff`), (2) processing state markers (`:Processed`, `:Indexed`, `:Reviewed`), (3) access control categories (`:PublicData`, `:ConfidentialData`). DO NOT use labels for: (1) values that change frequently — use properties instead, (2) more than approximately 50 distinct labels per graph — index performance degrades, (3) grouping nodes that have no structural difference in graph topology.

PART 2: Building Knowledge Graphs

From Structured Data, Unstructured Text, and Graph Design Patterns

2.1 From Structured Data (CSV / JSON)

The fastest path to a populated knowledge graph is importing structured data you already have: CSVs, JSON files, REST API responses, or database exports. Neo4j's built-in LOAD CSV command and APOC's apoc.load.json() handle both formats natively.

Golden rule: always create uniqueness constraints before loading data. Constraints prevent duplicate nodes during MERGE operations and implicitly create an index for fast lookups. Without them, MERGE degrades to a full label scan on every row.

LOAD CSV — Hotel Rooms

```
# Via Python neo4j driver — LOAD CSV
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'password')
)

def load_rooms_from_csv(csv_path):
    with driver.session() as s:
        s.run("""
LOAD CSV WITH HEADERS
FROM 'file:///rooms.csv' AS row
MERGE (r:Room {id: row.room_id})
SET r.floor = toInteger(row.floor),
    r.type = row.room_type,
    r.capacity = toInteger(row.capacity),
    r.rate_thb = toFloat(row.rate_thb)
""")

# CSV format: room_id,floor,room_type,capacity,rate_thb
```

apoc.load.json — IoT Sensor Readings

```
# JSON import with APOC
def load_iot_readings():
    with driver.session() as s:
        s.run("""
CALL apoc.load.json(
    'file:///iot_readings.json'
) YIELD value
MERGE (r:Room {id: value.room_id})
CREATE (reading:SensorReading {
    ts: datetime(value.timestamp),
    temp_c: value.temp,
    humidity: value.humidity,
    occupancy: value.occupied
})
CREATE (r)-[:HAS_READING]->(reading)
""")
```

Schema Constraints and Indexes

```
-- Always create constraints BEFORE loading data
CREATE CONSTRAINT room_id_unique
```

```

FOR (r:Room) REQUIRE r.id IS UNIQUE;

CREATE CONSTRAINT device_id_unique
FOR (d:Device) REQUIRE d.id IS UNIQUE;

CREATE INDEX room_floor_idx
FOR (r:Room) ON (r.floor);

CREATE INDEX reading_ts_idx
FOR (s:SensorReading) ON (s.ts);

```

2.2 From Unstructured Text (LLMGraphTransformer)

LLMGraphTransformer is a LangChain component that uses an LLM (Claude, GPT-4, Gemini) to extract entities and relationships from raw text. Feed it maintenance reports, incident logs, customer emails, or any unstructured document — it returns nodes and edges ready to insert into Neo4j. This is the foundation of the LLM Graph Builder pattern.

The key insight: you can define an extraction schema (allowed node labels and relationship types) to constrain what the LLM extracts. This keeps your graph clean and domain-focused, rather than producing a noisy open-information extraction result.

LLMGraphTransformer — Entity Extraction from Text

```

# pip install langchain-experimental langchain-anthropic
from langchain_experimental.graph_transformers import (
    LLMGraphTransformer
)
from langchain_community.graphs import Neo4jGraph
from langchain_anthropic import ChatAnthropic
from langchain_core.documents import Document

# Claude as the extraction LLM
llm = ChatAnthropic(
    model='claude-3-5-sonnet-20241022',
    api_key='YOUR_ANTHROPIC_KEY'
)

graph = Neo4jGraph(
    url='bolt://localhost:7687',
    username='neo4j',
    password='mas_memory_2024'
)

# Define extraction schema (optional but recommended)
transformer = LLMGraphTransformer(
    llm=llm,
    allowed_nodes=['Room', 'Device', 'Alert', 'Staff',
                  'GuestIncident', 'MaintenanceJob'],
    allowed_relationships=[
        'HAS_DEVICE', 'TRIGGERED', 'ASSIGNED_TO',
        'CAUSED_BY', 'RESOLVED_BY'
    ],
    node_properties=['name', 'type', 'status', 'timestamp'],
    relationship_properties=['since', 'severity', 'duration']
)

# Feed any text — maintenance reports, incident logs, etc.
docs = [
    Document(page_content=
        'Room 301 HVAC unit C-7 triggered a HIGH TEMP alert '
        'at 14:23. Maintenance technician Somchai was assigned '
        'and resolved the issue by replacing the filter. '

```

```
'Root cause: dirty air filter, recurrence risk: HIGH.'
)
]

# Extract entities and relationships
graph_docs = transformer.convert_to_graph_documents(docs)
graph.add_graph_documents(graph_docs, include_source=True)

print(f'Extracted: {len(graph_docs[0].nodes)} nodes, '
      f'{len(graph_docs[0].relationships)} relationships')
```

What Gets Created in Neo4j

After running this code, Neo4j contains: Room_301 → HVAC_C7 → Alert_HIGH_TEMP → Somchai → MaintenanceJob nodes, all connected with typed relationships (HAS_DEVICE, TRIGGERED, ASSIGNED_TO, RESOLVED_BY). The source Document node is also created and linked via :MENTIONS. This is the LLM Graph Builder pattern — any text becomes a traversable, queryable knowledge graph in seconds.

2.3 LLM Graph Builder (Docker Stack)

Neo4j Labs provides a full-stack LLM Graph Builder application with a React UI, Python backend, and Neo4j. Drop in PDF files, paste URLs, or link YouTube videos — the system extracts entities and relationships automatically using your choice of LLM.

The docker-compose below starts the complete stack locally. Set your API keys in a .env file before running.

docker-compose.yml — Full LLM Graph Builder Stack

```
version: "3.8"
services:
  neo4j:
    image: neo4j:5.18-community
    ports: ["7474:7474", "7687:7687"]
    environment:
      NE04J_AUTH: neo4j/mas_memory_2024
      NE04J_PLUGINS: '["apoc","graph-data-science"]'
    volumes: [neo4j_data:/data]

  llm-graph-builder-backend:
    image: neo4jlabs/llm-graph-builder-backend:latest
    ports: ["8000:8000"]
    environment:
      NE04J_URI: bolt://neo4j:7687
      NE04J_USERNAME: neo4j
      NE04J_PASSWORD: mas_memory_2024
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    depends_on: [neo4j]

  llm-graph-builder-frontend:
    image: neo4jlabs/llm-graph-builder-frontend:latest
    ports: ["3000:3000"]
    environment:
      BACKEND_API_URL: http://llm-graph-builder-backend:8000
    depends_on: [llm-graph-builder-backend]

volumes:
  neo4j_data:
```

After docker-compose up

Open <http://localhost:3000> for the React UI. Open <http://localhost:7474> for Neo4j Browser. Drop PDF files, point to URLs, or paste YouTube links — entities and relationships are extracted automatically. The backend calls your configured LLM (Anthropic or OpenAI) to perform extraction, then writes the results directly to Neo4j.

2.4 Domain Graph Design Patterns

Good knowledge graph design is not automatic — it requires deliberate choices about node granularity, relationship specificity, and temporal modelling. These patterns are battle-tested across production graph deployments.

Pattern	Description	Example	When to Use
Intermediate Node	Convert many-to-many into two one-to-many via a bridge node that carries relationship properties	Person-[:ENROLLED]->(Enrolment)-[:IN_COURSE]->(Course) — Enrolment holds grade, date	When relationship has multiple properties or needs to be queried independently
Typed Relationships	Use specific relationship types ([:TRIGGERS_ALERT]) not generic ones ([:IS_RELATED_TO])	(Device)-[:TRIGGERS_ALERT]->(Alert) vs (Device)-[:IS_RELATED_TO]->(Alert)	Always — relationship type is the primary filter in MATCH; generic types kill query performance
Timeline Graph	Link events chronologically with [:NEXT] edges to enable time-series traversal	(Event_1)-[:NEXT]->(Event_2)-[:NEXT]->(Event_3)	When order matters and you need efficient prev/next navigation
Bi-temporal	Track valid-time (when true in world) + transaction-time (when recorded in system) on nodes	Node properties: valid_from, valid_to, recorded_at, superseded_at	Compliance, auditing, slowly-changing dimensions
Qualified Relationships	Store numerical weight or metadata on relationship to enable weighted path algorithms	(Room)-[:ADJACENT_TO {distance_m: 12.5}]->(Room)	GDS shortest path, similarity scoring, cost-based routing
Singleton vs. Instance	One node for a type/category (RoomType:Suite) and separate nodes for each instance (Room:301)	(Room {id:301})-[:IS_TYPE]->(RoomType {name:Suite})	When you need class-level reasoning AND instance-level data
Anti-pattern: Generic Rels	NEVER use IS_RELATED_TO, HAS_CONNECTION, LINKED_TO — these make queries unscannable	Bad: (a)-[:IS_RELATED_TO]->(b) Good: (a)-[:TRIGGERS_ALERT]->(b)	This is always wrong — every relationship must have a specific semantic type

2.5 Knowledge Enrichment with GDS

After loading raw data, you can enrich the knowledge graph with computed properties using Graph Data Science algorithms. These enriched properties (importance scores, community IDs, embeddings) become first-class graph data that agents can query with Cypher.

The three enrichment algorithms below cover the most common production use cases: importance ranking (PageRank), clustering (Louvain), and structural embeddings (Node2Vec).

PageRank — Find Most Important Nodes

```
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
```



```

)

def compute_pagerank():
    with driver.session() as s:
        # Project the in-memory graph
        s.run("""
        CALL gds.graph.project(
            'deviceGraph',
            ['Device', 'Room', 'Alert'],
            ['HAS_DEVICE', 'TRIGGERED']
        )
        """)
        # Run PageRank, write scores back to nodes
        s.run("""
        CALL gds.pageRank.write('deviceGraph', {
            writeProperty: 'importance_score',
            maxIterations: 20
        })
        """)
        # Query most important devices
        result = s.run("""
        MATCH (d:Device)
        RETURN d.id, d.importance_score
        ORDER BY d.importance_score DESC LIMIT 5
        """)
        return list(result)

```

Louvain — Community Detection

```

def detect_communities():
    with driver.session() as s:
        s.run("""
        CALL gds.louvain.write('deviceGraph', {
            writeProperty: 'community_id'
        })
        """)
        result = s.run("""
        MATCH (d:Device)
        RETURN d.community_id,
        collect(d.id) AS devices
        ORDER BY size(collect(d.id)) DESC
        """)
        return list(result)

```

Node2Vec — Structural Graph Embeddings

```

def generate_node2vec_embeddings():
    with driver.session() as s:
        s.run("""
        CALL gds.node2vec.write('deviceGraph', {
            embeddingDimension: 64,
            writeProperty: 'n2v_embedding'
        })
        """)
        # After this, each Device node has n2v_embedding property
        # Use it for similarity search: cosine distance between nodes
        # based on graph topology, not text content

```

Key Insight:

After running these enrichment steps, your knowledge graph nodes carry computed properties: `importance_score` (PageRank), `community_id` (Louvain), and `n2v_embedding` (Node2Vec). Agents can query these directly: `"MATCH (d:Device) WHERE d.importance_score > 0.8 RETURN d.id"` — no separate ML pipeline call required at query time.

2.6 Vector Embeddings in Neo4j

Adding vector embeddings to your knowledge graph creates a hybrid semantic layer — every node can be found by either exact graph traversal (Cypher) or approximate semantic similarity (vector KNN). This combination is the core of production GraphRAG systems.

The workflow: (1) embed text properties using an embedding model, (2) store the embedding vector as a node property, (3) create a vector index over that property, (4) use KNN search to retrieve semantically similar nodes, (5) immediately follow with graph traversal to expand context.

Embedding Rooms and Semantic Search

```
# pip install langchain-openai
from langchain_openai import OpenAIEmbeddings
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

embedder = OpenAIEmbeddings(
    model='text-embedding-3-small', # 1536 dimensions
    api_key='YOUR_OPENAI_KEY'
)

def embed_room_descriptions():
    """Embed room descriptions and store in Neo4j."""
    with driver.session() as s:
        rooms = list(s.run("""
        MATCH (r:Room)
        WHERE r.embedding IS NULL
        AND r.description IS NOT NULL
        RETURN r.id AS id, r.description AS description
        """))
        texts = [row["description"] for row in rooms]
        embeddings = embedder.embed_documents(texts)
        for room, embedding in zip(rooms, embeddings):
            s.run("""
            MATCH (r:Room {id: $id})
            SET r.embedding = $embedding
            """, id=room["id"], embedding=embedding)
        print(f"Embedded {len(rooms)} rooms")

def semantic_room_search(query_text: str, k: int = 5):
    """Find rooms semantically similar to a text query."""
    query_embedding = embedder.embed_query(query_text)
    with driver.session() as s:
        results = s.run("""
        CALL db.index.vector.queryNodes(
            'room_embeddings', $k, $embedding
        ) YIELD node, score
        RETURN node.id, node.type, node.floor, score
        ORDER BY score DESC
        """, k=k, embedding=query_embedding)
    return list(results)

# Usage
rooms = semantic_room_search('quiet room with ocean view', k=3)
for r in rooms:
    print(f"Room {r['node.id']}: score={r['score']:.3f}")
```

Vector Index Creation and Hybrid Retrieval

```
-- Create vector index (run once after embeddings are computed)
CREATE VECTOR INDEX room_embeddings
FOR (r:Room) ON (r.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}}

-- Verify the index
SHOW VECTOR INDEXES

-- Hybrid GraphRAG: vector similarity + graph expansion
CALL db.index.vector.queryNodes(
  'room_embeddings', 3, $query_vec
) YIELD node AS room, score
MATCH (room)-[:HAS_DEVICE]->(d:Device)
MATCH (d)-[:TRIGGERED]->(a:Alert)
WHERE a.ts > datetime() - duration({days: 7})
RETURN room.id, room.type, score,
  collect(a.type) AS recent_alerts
ORDER BY score DESC
```

Key Insight: The vector + graph pattern returns results that pure vector search cannot: "Find rooms similar to my query AND show their recent device alerts." This is impossible with a standalone vector database but trivial in Neo4j — one Cypher query combines both retrieval modes with no extra round-trips.

2.7 Chunking Strategies for GraphRAG Documents

When building GraphRAG from documents, how you chunk the source text determines retrieval quality. Chunks too large lead to irrelevant context being included; chunks too small lose necessary semantic context. The right chunking strategy depends on your document type and query patterns.

The document graph pattern links chunks to their parent document and to each other via `:NEXT_CHUNK` relationships. This enables both random access (vector KNN to any chunk) and sequential expansion (retrieve chunk plus its immediate neighbours for context coherence).

Document Ingestion with Chunk Graph

```
# pip install langchain-text-splitters
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
  'bolt://localhost:7687',
  auth=('neo4j', 'mas_memory_2024')
)

def ingest_document(doc_text: str, doc_id: str,
  doc_title: str) -> int:
  """Chunk, embed, and store a document in Neo4j."""
  splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50
  )
  chunks = splitter.split_text(doc_text)
  embedder = OpenAIEmbeddings(model="text-embedding-3-small")
  embeddings = embedder.embed_documents(chunks)

  with driver.session() as s:
    s.run("""
```

```

MERGE (d:Document {id: $doc_id})
SET d.title = $title,
d.ingested_at = datetime()
"", doc_id=doc_id, title=doc_title)

prev_chunk_id = None
for i, (text, emb) in enumerate(zip(chunks, embeddings)):
    chunk_id = f"{doc_id}_chunk_{i}"
    s.run("""
CREATE (c:Chunk {
id: $chunk_id,
text: $text,
embedding: $embedding,
position: $pos,
doc_id: $doc_id
})
"", chunk_id=chunk_id, text=text,
embedding=emb, pos=i, doc_id=doc_id)
s.run("""
MATCH (d:Document {id: $doc_id}),
(c:Chunk {id: $chunk_id})
CREATE (d)-[:HAS_CHUNK]->(c)
"", doc_id=doc_id, chunk_id=chunk_id)
if prev_chunk_id:
    s.run("""
MATCH (prev:Chunk {id: $prev}),
(curr:Chunk {id: $curr})
CREATE (prev)-[:NEXT_CHUNK]->(curr)
"", prev=prev_chunk_id, curr=chunk_id)
prev_chunk_id = chunk_id
return len(chunks)

```

Chunking Strategy	Chunk Size	Overlap	Best For
Recursive Character Split	512 tokens	50 tokens	General documents, maintenance reports, emails
Sentence-level Split	3-5 sentences	1 sentence	FAQ documents, structured Q+A, technical manuals
Paragraph-level Split	1 paragraph	None	Articles, policies, procedures with clear structure
Fixed Token Split	256-1024 tokens	10-15%	Code, API docs, dense technical text
Semantic Split	Variable	Topic boundary	Mixed-content documents; requires embedding pass first

Window Retrieval with Context Expansion

```

-- Window retrieval: KNN match + adjacent chunk context
CALL db.index.vector.queryNodes(
'chunk_embeddings', 3, $query_vec
) YIELD node AS chunk, score
OPTIONAL MATCH (prev)-[:NEXT_CHUNK]->(chunk)
OPTIONAL MATCH (chunk)-[:NEXT_CHUNK]->(next)
OPTIONAL MATCH (chunk)-[:MENTIONS]->(entity)
RETURN chunk.text AS matched_chunk,
prev.text AS context_before,
next.text AS context_after,
collect(entity.name) AS entities,
score
ORDER BY score DESC

```

Key Insight: Window retrieval (fetching the chunk before and after the matched chunk) significantly improves context completeness without increasing k. Instead of setting k=10 to get 10 scattered chunks, set k=3 and expand each to a 3-chunk window — you get 9 chunks with guaranteed local coherence. This reliably improves Context Recall in RAGAS evaluation.

2.8 Production Neo4j Setup Checklist

Moving from a development Neo4j instance to production requires attention to security, performance, and observability. This section covers the essential production configuration checklist for teams deploying Neo4j as a knowledge graph backend.

Category	Setting / Action	Why It Matters
Security	Change default password immediately (neo4j/neo4j)	Default credentials are public knowledge — any exposed instance is compromised instantly
Security	Enable TLS for Bolt: dbms.connector.bolt.tls_level=REQUIRED	Bolt protocol transmits graph data in plaintext by default over the network
Security	Create role-based users: CREATE ROLE reader; GRANT READ ON GRAPH * TO reader	Principle of least privilege — agents should only have read access to the graph
Performance	Set heap: dbms.memory.heap.max_size=4G	Default 512MB is insufficient for GDS or large graph workloads
Performance	Set page cache: dbms.memory.pagecache.size=2G	Page cache holds the graph in memory — larger = fewer disk reads = faster queries
Performance	Enable slow query logging: dbms.logs.query.threshold=1000ms	Identify unindexed MATCH patterns before they cause production latency
Observability	Enable Prometheus metrics endpoint at port 2004	Monitor active transactions, memory, query latency in Grafana dashboards
Backup	Schedule neo4j-admin database dump daily to object storage	Neo4j Community has no hot backup — must stop the database for a consistent dump
Indexes	Review EXPLAIN for every Cypher template used in production	Missing indexes cause full label scans — catastrophic performance at any scale

Production Index and Constraint Setup

```
-- Complete production index and constraint setup
-- Run this script BEFORE any data loading

CREATE CONSTRAINT room_id_unique
FOR (r:Room) REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT device_id_unique
FOR (d:Device) REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT alert_id_unique
FOR (a:Alert) REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT chunk_id_unique
FOR (c:Chunk) REQUIRE c.id IS UNIQUE;

-- Performance indexes
CREATE INDEX room_floor_idx FOR (r:Room) ON (r.floor);
CREATE INDEX device_type_idx FOR (d:Device) ON (d.type);
CREATE INDEX alert_ts_idx FOR (a:Alert) ON (a.ts);
CREATE INDEX alert_severity_idx FOR (a:Alert) ON (a.severity);

-- Full-text search indexes
CREATE FULLTEXT INDEX room_fts
FOR (r:Room) ON EACH [r.description, r.type, r.id];
CREATE FULLTEXT INDEX device_fts
FOR (d:Device) ON EACH [d.model, d.type, d.id];
```

```
-- Vector index (after embeddings computed)
CREATE VECTOR INDEX chunk_embeddings
FOR (c:Chunk) ON (c.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}};

-- Verify all indexes created
SHOW INDEXES
```

PART 3: Applied GraphRAG with Major Frameworks

LangChain · LangGraph · CrewAI · Google ADK · MCP · LlamaIndex

GraphRAG (Graph Retrieval-Augmented Generation) combines the semantic richness of knowledge graphs with the generative power of large language models. Instead of treating retrieval as a simple vector lookup, GraphRAG uses graph traversal to expand context with connected entities, relationships, and structured facts — dramatically improving answer faithfulness and reducing hallucination.

This section shows how to implement GraphRAG with every major AI framework in 2025: LangChain (chains), LangGraph (ReAct agents), CrewAI (multi-agent crews), Google ADK (production agents), FastMCP (Claude tool server), and LlamaIndex (triple-store retrieval).

3.1 LangChain + Neo4j GraphRAG

LangChain provides two primary Neo4j integration patterns: GraphCypherQAChain (natural language → Cypher → answer) and Neo4jVector (semantic similarity search via Neo4j vector index). Both patterns work with any LLM that LangChain supports.

GraphCypherQAChain uses the schema discovered by `graph.refresh_schema()` as context for the LLM, enabling it to generate valid Cypher without manual schema injection. This means your graph schema is your API contract.

LangChain GraphCypherQAChain + Neo4jVector

```
# pip install langchain langchain-community langchain-anthropic
from langchain_community.graphs import Neo4jGraph
from langchain_community.vectorstores import Neo4jVector
from langchain.chains import GraphCypherQAChain
from langchain_anthropic import ChatAnthropic
from langchain_anthropic import AnthropicEmbeddings

# LLM and graph
llm = ChatAnthropic(
    model='claude-3-5-sonnet-20241022',
    api_key='YOUR_KEY'
)

graph = Neo4jGraph(
    url='bolt://localhost:7687',
    username='neo4j',
    password='mas_memory_2024'
)
graph.refresh_schema() # discovers labels, rel types, properties

# Pattern 1: GraphCypherQAChain (NL -> Cypher -> Answer)
chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
    verbose=True,
    return_intermediate_steps=True
)

result = chain.invoke({
    'query': 'Which rooms had temperature above 28C today?'
})
print('Answer:', result['result'])
print('Cypher:', result['intermediate_steps'][0]['query'])

# Pattern 2: Neo4jVector (semantic search)
vector_store = Neo4jVector.from_existing_index(
```

```

embedding=AnthropicEmbeddings(),
url='bolt://localhost:7687',
username='neo4j',
password='mas_memory_2024',
index_name='chunk_embeddings',
text_node_property='text'
)

# Similarity search returns Document objects
docs = vector_store.similarity_search(
'HVAC malfunction hotel room', k=3
)
for doc in docs:
print(doc.page_content[:100])

```

Key Insight: GraphCypherQACChain translates natural language to Cypher using the schema it discovers automatically via `graph.refresh_schema()`. This means your KG schema IS your API contract — well-designed schemas produce accurate Cypher. Use `return_intermediate_steps=True` to see the generated Cypher for debugging and prompt tuning.

3.2 LangGraph + Neo4j (ReAct Agent)

LangGraph is LangChain's framework for building stateful, graph-structured AI agents. The `create_react_agent` helper creates a ReAct (Reason + Act) agent that can use tools to query and write to the knowledge graph. The agent's reasoning loop is itself a graph (LangGraph) operating on a knowledge graph (Neo4j) — graphs all the way down.

The `@tool` decorator transforms any Python function into an LLM-callable tool. The docstring becomes the tool description that the LLM uses to decide when and how to call the tool — write it carefully.

LangGraph ReAct Agent with KG Tools

```

# pip install langgraph langchain-anthropic
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool
from langchain_anthropic import ChatAnthropic
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
'bolt://localhost:7687',
auth=('neo4j', 'mas_memory_2024')
)

@tool
def query_knowledge_graph(cypher: str) -> str:
    """
    Execute a Cypher query on the hotel knowledge graph.
    Use for: room status, device readings, incident history,
    energy consumption, guest patterns.
    Args:
    cypher: valid Cypher query string
    """
    with driver.session() as s:
        result = s.run(cypher)
        rows = [dict(r) for r in result]
        return str(rows[:10]) # limit output

@tool
def store_agent_event(
    agent_id: str,
    event_type: str,
    details: str

```



```

) -> str:
"""Store an agent action in the knowledge graph."""
with driver.session() as s:
s.run("""
MERGE (a:Agent {id: $aid})
CREATE (e:Event {
type: $etype,
details: $details,
ts: datetime()
})
CREATE (a)-[:PERFORMED]->(e)
""", aid=agent_id, etype=event_type,
details=details)
return f'Stored: {event_type}'

# ReAct agent with KG tools
llm = ChatAnthropic(
model='claude-3-5-sonnet-20241022',
api_key='YOUR_ANTHROPIC_KEY'
)

agent = create_react_agent(
model=llm,
tools=[query_knowledge_graph, store_agent_event],
prompt=(
'You are a hotel energy management AI. '
'Use the knowledge graph to answer questions '
'about room status, energy consumption, and '
'maintenance history. Always store your key '
'findings as events.'
)
)

# Run the agent
result = agent.invoke({
'messages': [{
'role': 'user',
'content': 'Which rooms on floor 3 have '
'had HVAC issues in the last 7 days?'
}]
})
print(result['messages'][-1].content)

```

Key Insight: The `@tool` decorator is all that is needed to give LangGraph agents access to your knowledge graph. The agent uses ReAct (Reason + Act) to decide when to query the graph and how to interpret results — true agentic GraphRAG. The `store_agent_event` tool creates a persistent audit trail of every agent decision directly in the knowledge graph.

3.3 CrewAI + Neo4j (Multi-Agent Crew)

CrewAI orchestrates multiple specialized agents as a "crew" — each agent has a role, goal, backstory, and set of tools. When combined with Neo4j, the knowledge graph becomes a shared blackboard where agents read findings from each other and store their own outputs.

The pattern below demonstrates a two-agent crew: an Analyst that reads energy data from the KG, and an Optimizer that writes recommendations back as OptimizationAction nodes. This is emergent collective intelligence via graph-shared state.

CrewAI Multi-Agent Crew with Shared Neo4j KG

```

# pip install crewai
from crewai import Agent, Task, Crew, Process

```

```

from crewai_tools import BaseTool
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

class KnowledgeGraphTool(BaseTool):
    name: str = 'knowledge_graph_query'
    description: str = (
        'Query the hotel knowledge graph using Cypher. '
        'Use for room status, device data, incident history.'
    )

    def _run(self, cypher: str) -> str:
        with driver.session() as s:
            result = s.run(cypher)
            return str([dict(r) for r in result][:5])

class GraphMemoryTool(BaseTool):
    name: str = 'store_to_graph'
    description: str = (
        'Store an insight or finding in the knowledge graph.'
    )

    def _run(self, label: str,
              properties: str, relationship: str = '') -> str:
        with driver.session() as s:
            s.run(
                f'MERGE (n:{label} {{data: $props, '
                f'ts: datetime()}}) ',
                props=properties
            )
            return f'Stored {label} node'

# Agents with graph tools
analyst = Agent(
    role='Hotel Energy Analyst',
    goal='Analyse energy patterns using knowledge graph data',
    backstory='Data scientist specialising in hotel IoT analytics',
    tools=[KnowledgeGraphTool()],
    llm='claude-3-5-sonnet-20241022',
    verbose=False
)

optimizer = Agent(
    role='Energy Optimizer',
    goal='Recommend and store optimization actions in KG',
    backstory='Energy engineer with graph-based decision making',
    tools=[KnowledgeGraphTool(), GraphMemoryTool()],
    llm='claude-3-5-sonnet-20241022',
    verbose=False
)

# Tasks
analyse_task = Task(
    description=(
        'Query the KG: find all rooms where average '
        'energy consumption > 3 kWh in the last 24 hours. '
        'List room IDs and consumption values.'
    ),
    expected_output='List of high-consumption rooms',

```

```

agent=analyst
)

optimize_task = Task(
description=(
'For high-consumption rooms found, recommend '
'HVAC setpoint adjustments. Store recommendations '
'as OptimizationAction nodes in the KG.'
),
expected_output='Dict of room to recommended action',
agent=optimizer
)

crew = Crew(
agents=[analyst, optimizer],
tasks=[analyse_task, optimize_task],
process=Process.sequential,
verbose=False
)

result = crew.kickoff()
print(result)

```

Key Insight: CrewAI agents share a common knowledge graph — when one agent stores a finding, another can immediately query it. This creates emergent collective intelligence where the KG is the shared blackboard. The sequential process ensures the Optimizer sees the Analyst's results; for parallel work, switch to `Process.hierarchical` with a manager agent.

3.4 Google ADK + Neo4j

Google Agent Development Kit (ADK) is Google's production-grade framework for building Gemini-powered agents. It uses plain Python functions as tools via `FunctionTool` — write a function with a clear docstring, and ADK's Gemini model decides when and how to call it.

ADK excels at production deployments: it integrates with Google Cloud Run, Vertex AI, and Agent Engine. The `InMemorySessionService` is used for development; swap it for a persistent session service in production to maintain conversation state across requests.

Google ADK Agent with Neo4j Tools

```

# pip install google-adk
from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

# Tool functions (plain Python — no decorator needed)
def query_hotel_graph(cypher: str) -> dict:
    """
    Query the hotel knowledge graph with a Cypher query.
    Use for finding rooms, devices, events, patterns.
    Returns: dict with 'results' list and 'count' int.
    """
    with driver.session() as s:
        rows = [dict(r) for r in s.run(cypher)]
        return {'results': rows[:10], 'count': len(rows)}

```

```

def find_related_entities(
    entity_type: str,
    entity_name: str,
    relationship: str,
    hops: int = 1
) -> dict:
    """
    Find all entities related to a given entity via
    a specified relationship type.
    Args:
    entity_type: Neo4j label (e.g. 'Room', 'Device')
    entity_name: entity identifier or name
    relationship: Cypher relationship type (e.g. 'TRIGGERS')
    hops: max traversal depth (1-3)
    """
    cypher = (
        f'MATCH (n:{entity_type} {{name: $name}})'
        f'-[:{relationship}*1..{hops}]->(related) '
        f'RETURN related.name, labels(related)'
    )
    with driver.session() as s:
        rows = [dict(r) for r in s.run(cypher, name=entity_name)]
    return {'related': rows}

# ADK Agent with graph tools
hotel_agent = Agent(
    name='hotel_knowledge_agent',
    model='gemini-2.0-flash',
    instruction=(
        'You are a hotel intelligence agent with access to '
        'a Neo4j knowledge graph containing room status, '
        'device readings, maintenance history, and guest data. '
        'Use graph queries to provide accurate, evidence-based '
        'answers to hotel operations questions.'
    ),
    tools=[
        FunctionTool(query_hotel_graph),
        FunctionTool(find_related_entities)
    ]
)

# Runner setup
session_service = InMemorySessionService()
runner = Runner(
    agent=hotel_agent,
    app_name='HotelKnowledgeAgent',
    session_service=session_service
)

if __name__ == '__main__':
    print('Hotel knowledge agent ready.')
    print("Query: 'Which devices triggered alerts today?'")

```

Key Insight: Google ADK's FunctionTool accepts any plain Python function — just write the function with a clear docstring and ADK's Gemini model decides when and how to call it. The function signature IS the tool's API contract. Type hints on parameters are used to generate the JSON schema that Gemini uses for structured function calling — always include them.

3.5 GraphRAG MCP Server (Model Context Protocol)

Model Context Protocol (MCP) is Anthropic's open standard for connecting AI models to external tools and data sources. Building an MCP server over Neo4j turns your knowledge graph into a tool that Claude can use directly in Claude Desktop or via the Claude API — no LangChain or ADK required.

FastMCP is the Python framework for building MCP servers. The `@mcp.tool()` decorator registers a function as an MCP tool; the `@mcp.resource()` decorator registers static or dynamic data that Claude can read as context. Together they create a complete MCP interface over your graph.

FastMCP — Neo4j MCP Server for Claude Desktop

```
# pip install fastmcp neo4j
# Build a Model Context Protocol server for Claude Desktop

from fastmcp import FastMCP
from neo4j import GraphDatabase
import json

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

mcp = FastMCP('Hotel Knowledge Graph MCP Server')

@mcp.tool()
def query_graph(cypher: str) -> str:
    """
    Execute a Cypher query against the hotel knowledge graph.
    The graph contains: Room, Device, Alert, Event, Agent,
    Supplier, Staff, Guest nodes.
    Use standard Cypher syntax.
    """
    with driver.session() as s:
        result = s.run(cypher)
        rows = [dict(r) for r in result]
        return json.dumps(rows[:20], default=str)

@mcp.tool()
def get_graph_schema() -> str:
    """Return the current knowledge graph schema."""
    with driver.session() as s:
        labels = [r[0] for r in s.run('CALL db.labels()')]
        rels = [r[0] for r in s.run('CALL db.relationshipTypes()')]
        return json.dumps({
            'node_labels': labels,
            'relationship_types': rels
        })

@mcp.tool()
def find_anomalies(hours: int = 24) -> str:
    """Find all anomalies detected in the last N hours."""
    with driver.session() as s:
        result = s.run("""
        MATCH (e:Event {type: 'anomaly'})
        WHERE e.ts > datetime() - duration({hours: $h})
        MATCH (e)-[:INVOLVES]->(n)
        RETURN e.severity, n.name, e.ts
        ORDER BY e.ts DESC LIMIT 20
        """, h=hours)
    return json.dumps(
        [dict(r) for r in result], default=str
    )
```

```

@mcp.resource('graph://schema')
def graph_schema_resource() -> str:
    """Current Neo4j schema as a resource."""
    return get_graph_schema()

if __name__ == '__main__':
    mcp.run()

# Add to Claude Desktop config:
# ~/.claude/claude_desktop_config.json
# {
#   "mcpServers": {
#     "hotel-kg": {
#       "command": "python",
#       "args": ["/path/to/this/mcp_server.py"]
#     }
#   }
# }

```

Key Insight: An MCP server turns your Neo4j knowledge graph into a tool Claude can use directly in Claude Desktop or Claude API. Any query Claude would answer from memory is now answered from your verified, up-to-date graph data. The `@mcp.resource` decorator exposes the schema so Claude always knows what node labels and relationship types are available before writing Cypher queries.

3.6 LlamaIndex + Neo4j (Triple-Store Retrieval)

LlamaIndex provides a `KnowledgeGraphIndex` that automatically extracts (subject, predicate, object) triples from documents and stores them in Neo4j. This is the triple-store approach — simpler than `LLMGraphTransformer` but excellent for document-centric knowledge bases where you want automatic extraction without defining a schema.

LlamaIndex supports three retrieval modes: keyword (BM25 on entity names), embedding (cosine similarity), and hybrid (both). The hybrid mode is recommended for production — it handles both exact entity lookups and semantic paraphrases.

LlamaIndex KnowledgeGraphIndex with Neo4j

```

# pip install llama-index llama-index-graph-stores-neo4j
from llama_index.core import (
    KnowledgeGraphIndex,
    StorageContext,
    Settings
)
from llama_index.graph_stores.neo4j import Neo4jGraphStore
from llama_index.llms.anthropic import Anthropic
from llama_index.core import SimpleDirectoryReader

# Configure Claude as LLM
Settings.llm = Anthropic(
    model='claude-3-5-sonnet-20241022',
    api_key='YOUR_ANTHROPIC_KEY'
)

# Connect to Neo4j
graph_store = Neo4jGraphStore(
    username='neo4j',
    password='mas_memory_2024',
    url='bolt://localhost:7687',
    database='neo4j'
)

storage_context = StorageContext.from_defaults(
    graph_store=graph_store
)

```

```

)

# Build index from documents (auto-extracts triples)
documents = SimpleDirectoryReader(
    'hotel_reports/'
).load_data()

index = KnowledgeGraphIndex.from_documents(
    documents,
    storage_context=storage_context,
    max_triplets_per_chunk=10,
    include_embeddings=True,
    show_progress=True
)

# Query the knowledge graph
query_engine = index.as_query_engine(
    include_text=True,
    retriever_mode='keyword', # or 'embedding' or 'hybrid'
    response_mode='tree_summarize'
)

response = query_engine.query(
    'What were the main causes of HVAC failures '
    'in hotel rooms during Q1 2024?'
)
print(response)

# View the extracted graph structure
print('\nGraph triples extracted:')
for triple in index.get_networkx_graph().edges(data=True):
    print(f' {triple[0]} -> {triple[2].get("label","")} -> {triple[1]}')

```

Key Insight: LlamaIndex auto-extracts (subject, predicate, object) triples from documents and stores them in Neo4j. This is triple-store style — simpler than LLMGraphTransformer but great for document-centric knowledge bases. The `get_networkx_graph()` call lets you visualise the extracted graph using matplotlib or pyvis for debugging and verification.

Framework Comparison Summary

Framework	Best For	Neo4j Integration	LLM Support
LangChain	Chains, RAG pipelines, prototype to production	Neo4jGraph + Neo4jVector + GraphCypherQACchain	All major LLMs via LangChain providers
LangGraph	Stateful ReAct agents, multi-step reasoning, human-in-loop	@tool functions wrapping neo4j driver	All LangChain-compatible LLMs
CrewAI	Multi-agent teams, role-based collaboration, autonomous crews	BaseTool subclass wrapping neo4j driver	Claude, GPT-4, Gemini via CrewAI llm param
Google ADK	Production Gemini agents, Google Cloud deployment, enterprise	FunctionTool wrapping plain Python neo4j functions	Gemini 2.0 (primary), others via LiteLLM
FastMCP	Claude Desktop tools, API-as-graph, schema-aware tool calling	@mcp.tool() and @mcp.resource() decorators	Claude (native); others via MCP proxies

Framework	Best For	Neo4j Integration	LLM Support
LlamaIndex	Document ingestion, triple extraction, RAG over text corpora	Neo4jGraphStore + KnowledgeGraphIndex	All LlamaIndex-compatible LLMs

3.7 GraphRAG Architecture Patterns

There are three primary GraphRAG architecture patterns. Choosing the right pattern for your use case determines both the quality of responses and the complexity of implementation. These are not mutually exclusive — production systems often combine all three in a layered pipeline.

Pattern	How It Works	Best For	Complexity
NL-to-Cypher (Text2Cypher)	LLM converts natural language question to Cypher; executes it; generates answer from structured result	Structured questions about entities, counts, aggregations; business intelligence over graph data	Medium — requires good schema prompting and Cypher validation layer
Vector + Graph Expand	Vector KNN finds semantically similar chunks; Cypher traversal expands to connected graph nodes	Document QA where chunks need enrichment from connected structured graph facts	Low-Medium — straightforward to implement with any vector store + Neo4j
Agent with KG Tools	ReAct or planning agent decides which KG tool to call; chains multiple graph operations iteratively	Complex multi-step reasoning; queries requiring iterative graph exploration and memory	High — requires careful tool design and prompt engineering; test all edge cases

Custom Text2Cypher Pipeline

Text2Cypher is the most reliable GraphRAG pattern for structured questions. The LLM generates a Cypher query using the graph schema as context, executes it, then formats the structured result into a natural language answer. Temperature should be 0 for deterministic Cypher.

```
Custom Text2Cypher Pipeline

from langchain_anthropic import ChatAnthropic
from langchain_community.graphs import Neo4jGraph
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatAnthropic(
    model='claude-3-5-sonnet-20241022',
    temperature=0 # deterministic Cypher generation
)

graph = Neo4jGraph(
    url='bolt://localhost:7687',
    username='neo4j', password='mas_memory_2024'
)
schema = graph.get_schema

cypher_prompt = ChatPromptTemplate.from_messages([
    ('system',
     "Convert the user question to Cypher using this schema:\n"
     "{schema}\n"
```



```

"Rules: always add LIMIT 20; use OPTIONAL MATCH for optional"
" relationships; return only the Cypher query."),
('human', '{question}')
])

answer_prompt = ChatPromptTemplate.from_messages([
('system',
"Answer using ONLY the data provided. "
"Do not add information beyond the query results."),
('human', 'Question: {question}\nData: {data}\nAnswer:')
])

def text2cypher_query(question: str) -> dict:
cypher = (cypher_prompt | llm | StrOutputParser()).invoke({
'schema': schema, 'question': question
})
data = graph.query(cypher)
answer = (answer_prompt | llm | StrOutputParser()).invoke({
'question': question, 'data': str(data)
})
return {'cypher': cypher, 'data': data, 'answer': answer}

result = text2cypher_query(
'How many rooms had energy consumption above 5 kWh yesterday?'
)
print('Cypher:', result['cypher'])
print('Answer:', result['answer'])

```

Key Insight: Always validate Cypher before executing it in production. Wrap `graph.query()` in a try-except block; on `SyntaxError`, either retry with a corrected prompt or fall back to a vector search. A Cypher error should never surface as an unhandled exception to the end user — degrade gracefully.

3.8 Agentic Memory Architecture with Neo4j

The most powerful use of Neo4j in agentic AI is as a persistent agent memory store. Agents accumulate knowledge over time: facts they have learned, actions they have taken, entities they have encountered. A knowledge graph stores all of this in a queryable, traversable form that survives session boundaries.

The four-layer memory architecture maps directly to Neo4j node types: Working memory (current session Python dict — not persisted), Episodic memory (:Event nodes with timestamps — what happened), Semantic memory (:Fact nodes with embeddings — what is true), Procedural memory (:Procedure nodes — how to do things).

AgentMemory — Neo4j-Backed Persistent Memory

```

from neo4j import GraphDatabase
import uuid

class AgentMemory:
    """Neo4j-backed persistent memory for AI agents."""

    def __init__(self, agent_id: str):
        self.agent_id = agent_id
        self.driver = GraphDatabase.driver(
            'bolt://localhost:7687',
            auth=('neo4j', 'mas_memory_2024')
        )
        self._ensure_agent_node()

    def _ensure_agent_node(self):
        with self.driver.session() as s:
            s.run("""
MERGE (a:Agent {id: $aid})

```

```

ON CREATE SET a.created = datetime()
"", aid=self.agent_id)

def remember_event(self, event_type: str,
details: dict) -> str:
    """Store an episodic memory (past action or observation)."""
    event_id = str(uuid.uuid4())[:8]
    with self.driver.session() as s:
        s.run("""
        MATCH (a:Agent {id: $aid})
        CREATE (e:Event {
        id: $eid, type: $etype,
        ts: datetime(), details: $details
        })
        CREATE (a)-[:PERFORMED]->(e)
        """, aid=self.agent_id, eid=event_id,
        etype=event_type, details=str(details))
    return event_id

def recall_recent(self, n: int = 10) -> list:
    """Retrieve N most recent episodic memories."""
    with self.driver.session() as s:
        result = s.run("""
        MATCH (a:Agent {id: $aid})-[:PERFORMED]->(e:Event)
        RETURN e.type AS type, e.ts AS ts,
        e.details AS details
        ORDER BY e.ts DESC LIMIT $n
        """, aid=self.agent_id, n=n)
    return [dict(r) for r in result]

def store_fact(self, subject: str, predicate: str,
obj: str, confidence: float = 1.0):
    """Store a semantic memory (learned fact as triple)."""
    with self.driver.session() as s:
        s.run("""
        MERGE (s:Entity {name: $subject})
        MERGE (o:Entity {name: $object})
        MERGE (s)-[:FACT {predicate: $pred}]->(o)
        SET r.confidence = $conf,
        r.learned_by = $aid,
        r.learned_at = datetime()
        """, subject=subject, pred=predicate,
        object=obj, conf=confidence,
        aid=self.agent_id)

# Usage
memory = AgentMemory('energy_optimizer_v1')
memory.remember_event('ANALYSIS', {
    'rooms_checked': 45, 'anomalies_found': 3
})
memory.store_fact('Room_301', 'HAS_ISSUE', 'HVAC_Overheating',
confidence=0.95)
recent = memory.recall_recent(5)
print(recent)

```

Key Insight:

The AgentMemory class gives any agent — LangGraph, CrewAI, Google ADK, or custom — a persistent Neo4j-backed memory with zero framework dependencies. Episodes are stored as Event nodes; semantic facts as Entity-FACT-Entity triples. The agent full history is queryable with Cypher at any time, enabling cross-session reasoning and learning from past mistakes.

PART 4: Evaluating GraphRAG Systems

RAGAS Metrics · Failure Modes · Improvement Loop

Building a GraphRAG system is only half the work — you must measure whether it actually works. RAGAS (Retrieval-Augmented Generation Assessment) is the standard evaluation framework for RAG systems. It measures four orthogonal dimensions of quality: faithfulness, answer relevancy, context precision, and context recall.

RAGAS uses an LLM as a judge — it evaluates the quality of responses by comparing them against ground truth answers and retrieved contexts. This LLM-as-judge approach scales to thousands of test cases without human annotation, enabling automated regression testing of GraphRAG pipelines.

4.1 RAGAS Metrics Overview

Metric	What It Measures	Formula (simplified)	Good Score
Faithfulness	Is the answer grounded in the retrieved context? Penalises hallucinated claims.	# claims in answer supported by context / # total claims in answer	> 0.90
Answer Relevancy	Is the answer actually relevant to the question asked? Penalises off-topic responses.	cosine_sim(question, reverse-engineered questions from answer)	> 0.85
Context Precision@K	Are the relevant chunks ranked first? Penalises systems that bury relevant context.	Weighted mean of Precision@k for k=1..K, weighted by relevance indicator v_k	> 0.80
Context Recall	Did retrieval find all necessary context? Penalises systems that miss key facts.	# ground truth claims supported by context / # total ground truth claims	> 0.75

RAGAS — Context Precision@K Formula

Context Precision@K = $(1/|\text{relevant items}|) * \sum_{k=1..K} [\text{Precision}@k * v_k]$, where $\text{Precision}@k = (\text{true_positives}@k) / (\text{true_positives}@k + \text{false_positives}@k)$ and $v_k \in \{0,1\}$ is the relevance indicator for the k-th item. This metric penalises systems that retrieve relevant chunks but rank irrelevant chunks above them. A system with Context Precision@K=1.0 always ranks relevant chunks first. Compare with MAP (Mean Average Precision) in information retrieval — Context Precision@K is its RAG-specific adaptation.

4.2 RAGAS Evaluation Pipeline

The RAGAS evaluation pipeline requires four inputs for each test case: question (what the user asked), ground_truth (correct answer), contexts (what was retrieved), and answer (what the system generated). In production, generate test datasets using RAGAS's TestsetGenerator from your knowledge graph documents.

RAGAS Evaluation Pipeline

```
# pip install ragas langchain-anthropic datasets
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall,
)
from ragas.llms import LangchainLLMWrapper
from langchain_anthropic import ChatAnthropic
from datasets import Dataset
```

```

# Setup evaluator LLM (Claude as judge)
eval_llm = LangchainLLMWrapper(
    ChatAnthropic(
        model='claude-3-5-sonnet-20241022',
        api_key='YOUR_ANTHROPIC_KEY'
    )
)

# Test dataset: questions, ground truths, contexts, answers
# In production: generate from your KG using TestsetGenerator
test_data = {
    'question': [
        'Which rooms had HVAC issues last week?',
        'What is the energy consumption of Room 301?',
    ],
    'ground_truth': [
        'Rooms 301, 205, and 412 had HVAC issues last week.',
        'Room 301 consumed 3.2 kWh on average last week.',
    ],
    'contexts': [
        [
            'Room 301 HVAC triggered alert on Monday.',
            'Room 205 had maintenance on Tuesday.',
            'Room 412 reported cold air on Wednesday.'
        ],
        [
            'Room 301 sensor reading: 3.2 kWh, temp 24C.'
        ],
    ],
    'answer': [
        'Rooms 301, 205, and 412 experienced HVAC problems.',
        'Room 301 used approximately 3.2 kWh last week.',
    ],
}

dataset = Dataset.from_dict(test_data)

# Evaluate all four RAGAS metrics
results = evaluate(
    dataset=dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall,
    ],
    llm=eval_llm,
)

print(results.to_pandas()[[
    'faithfulness', 'answer_relevancy',
    'context_precision', 'context_recall'
]])

```

Interpreting RAGAS Output

RAGAS returns a score between 0.0 and 1.0 for each metric per test case, plus dataset averages. A faithfulness score of 0.7 means 30% of claims in the answer are NOT supported by the retrieved context — the system is hallucinating. Start your improvement efforts with the lowest-scoring metric. Track scores across iterations in a simple JSON log file to detect regressions.

4.3 Failure Mode Taxonomy

GraphRAG systems fail in predictable ways. Knowing the taxonomy lets you diagnose failures from symptoms without manual debugging of every case.

Failure Mode	Root Cause	Symptom	Fix
Cypher Syntax Error	Schema mismatch or LLM hallucinating node labels / property names	Query fails with <code>SyntaxError</code> or <code>ClientError</code> in Neo4j	Add schema examples to system prompt; use <code>CALL db.schema.visualization()</code> to refresh
Hallucinated Entities	LLM invents node names or IDs that do not exist in the graph	Answer references non-existent rooms, devices, or staff members	Add entity list to system prompt; use Neo4j full-text search to resolve names first
Wrong Relationship Direction	LLM reverses edge direction in Cypher (e.g., <code>MATCH (a)-[:TRIGGERS]-(b)</code> when <code>b</code> triggers <code>a</code>)	Empty result set from a syntactically valid query	Always include direction examples in schema; add <code>EXPLAIN</code> to prompt; test both directions
Missing Context	Retrieval miss — the relevant chunk or node was not returned by vector search or Cypher	Correct Cypher generates empty results; agent answers from LLM memory instead	Add fallback vector search; improve embedding coverage; reduce chunk size; add synonyms
Over-Retrieval	Too many hops in Cypher or too large a <code>k</code> in vector search	Context window overflow; very slow queries; irrelevant context drowning relevant chunks	Add <code>LIMIT</code> to all <code>MATCH</code> clauses; use <code>k=3</code> not <code>k=20</code> for vector search; add hop limits
Low Faithfulness	LLM adds knowledge beyond retrieved context (world knowledge leakage)	Answer contains specific facts not present in any retrieved chunk or graph result	Reduce LLM temperature to 0; add "only use provided context" instruction; use smaller models

4.4 The GraphRAG Improvement Loop

GraphRAG quality improvement is an iterative engineering process, not a one-time setup. Each iteration follows a structured loop: evaluate metrics, diagnose the dominant failure mode, apply a targeted fix, and measure improvement. Expect 3-5 iterations to reach production quality.

GraphRAG Improvement Loop (Iterative Engineering)

```
+-----+
| GraphRAG Improvement Loop |
+-----+

1. DESIGN Define KG schema + domain ontology
| Node labels, rel types, properties
v
2. BUILD Load data, extract entities, enrich
| CSV/JSON/LLMGraphTransformer/APOC
v
3. EVALUATE Run RAGAS on test set (target: F>0.9)
| faithfulness, context_recall, precision
v
4. DIAGNOSE Identify failure mode from taxonomy
| Low faithfulness? Hallucinated entities?
v
5. IMPROVE Schema refinement / prompt update
| Add constraints, fix Cypher templates
v
```

```
6. REPEAT Track metric trends across iterations
| JSON log: {iter, faithfulness, recall}
+-----> back to EVALUATE
```

Iteration Tracking Template

Iteration	Faithfulness	Answer Relevancy	Context Precision	Context Recall	Action Taken
Baseline	0.61	0.72	0.58	0.54	No prompt engineering; raw GraphCypherQChain
Iter 1	0.74	0.79	0.66	0.61	Added schema examples to system prompt; set temperature=0
Iter 2	0.82	0.84	0.75	0.70	Added entity list; fallback vector search on Cypher error
Iter 3	0.91	0.88	0.83	0.79	Hybrid retrieval; LIMIT 5 on all MATCH; hop limit 3
Target	>0.90	>0.85	>0.80	>0.75	Production-ready threshold

Key Insight: A GraphRAG system with faithfulness > 0.9 and context_recall > 0.75 is production-ready for enterprise use. Below these thresholds, the system will hallucinate or miss critical information — both unacceptable for IoT/medical/financial domains. Three iterations of targeted improvements typically move a baseline system from 0.6 to 0.9 faithfulness. Document every change in a structured iteration log.

Course Summary — Knowledge Graph Mastery Checklist

After completing this course, you should be able to check off all items in the mastery checklist below. Use this as a self-assessment before your Week 9 project submission.

Domain	Skill	Evidence
KG Fundamentals	Explain nodes, edges, properties vs RDBMS and document DB	Can compare data models without notes
Neo4j Architecture	Explain index-free adjacency and why it enables fast multi-hop	Can draw the architecture from memory
Cypher Basics	Write CREATE, MATCH, MERGE, WHERE, WITH queries	Completes Section 1.6 exercises without hints
Cypher Intermediate	Use multi-hop paths, CALL subqueries, pattern comprehensions	Solves advanced exercises in < 5 minutes
Cypher Advanced	Create vector indexes, run GDS algorithms, use APOC	Can enrich a graph with PageRank and embeddings
Data Loading	Load CSV with constraints, use LLMGraphTransformer	Has run both import patterns on the hotel dataset
LLM Graph Builder	Deploy Docker stack, extract from PDF/URL	Has extracted 50+ nodes from a real document
Graph Design	Apply intermediate node, typed rels, timeline patterns	Can critique and redesign a poorly-designed schema
LangChain	Build GraphCypherQChain + Neo4jVector pipeline	Has a working NL-to-graph query chain

Domain	Skill	Evidence
LangGraph	Create ReAct agent with KG query and write tools	Agent correctly answers and stores 3+ hotel queries
CrewAI	Build multi-agent crew sharing a Neo4j knowledge graph	Analyst + Optimizer crew runs end-to-end
Google ADK	Build FunctionTool-based agent with Gemini	ADK agent correctly uses query_hotel_graph tool
MCP Server	Build FastMCP server, configure Claude Desktop	Claude Desktop can query hotel KG via MCP
LlamaIndex	Build KnowledgeGraphIndex from documents	Query engine returns correct answers from triples
RAGAS	Run all four RAGAS metrics on a test dataset	Has baseline scores for own GraphRAG system
Improvement Loop	Apply 3 iterations of targeted improvement	Faithfulness improved from baseline to > 0.85

Closing Thought — Graphs as Agent Cognition Substrate

Woolridge's seminal insight is that intelligent agents need a structured, queryable belief store. After this course, you have the tools to build that belief store as a production Neo4j knowledge graph — one that agents can read, write, traverse, and reason over. The combination of graph traversal (Cypher), semantic similarity (vector index), graph ML (GDS), and LLM reasoning (GraphRAG) is the state of the art in grounded AI agents as of 2025. GraphRAG is not a technique — it is an architecture. Build it deliberately.

4.5 Automated Test Dataset Generation

Manually creating test datasets is expensive and slow. RAGAS provides a TestsetGenerator that automatically generates question-answer pairs from your documents using an LLM. This enables creating hundreds of evaluation examples in minutes, covering three question types.

RAGAS generates: Simple questions (single-hop, directly answerable from one chunk), Reasoning questions (require inference across multiple chunks), and Multi-context questions (require information from multiple documents). A balanced test set should include all three types.

RAGAS TestsetGenerator

```
# pip install ragas langchain-anthropic
from ragas.testset import TestsetGenerator
from ragas.testset.evolutions import (
    simple, reasoning, multi_context
)
from langchain_anthropic import ChatAnthropic
from langchain_community.document_loaders import DirectoryLoader

# Load source documents
loader = DirectoryLoader('hotel_docs/', glob='**/*.txt')
documents = loader.load()

# Configure generator LLMs
generator_llm = ChatAnthropic(
    model='claude-3-5-sonnet-20241022'
)
critic_llm = ChatAnthropic(
    model='claude-3-5-sonnet-20241022'
)

# Create test set generator
```

```

generator = TestsetGenerator.from_langchain(
    generator_llm=generator_llm,
    critic_llm=critic_llm
)

# Generate 50 test cases
testset = generator.generate_with_langchain_docs(
    documents=documents,
    test_size=50,
    distributions={
        simple: 0.4, # 40% single-hop questions
        reasoning: 0.4, # 40% multi-step reasoning
        multi_context: 0.2 # 20% cross-document synthesis
    }
)

df = testset.to_pandas()
print(df[['question', 'ground_truth', 'question_type']].head())
df.to_csv('hotel_kg_testset.csv', index=False)
print(f'Generated {len(df)} test cases')

```

Test Set Quality

Review generated questions before using them for evaluation — LLMs sometimes generate ambiguous or unanswerable questions. Filter out: (1) questions that reference information not in your graph, (2) questions with multiple valid answers that RAGAS might score incorrectly, (3) yes/no questions with low discrimination value. A quality test set of 30 to 50 well-chosen questions is more valuable than 500 poor ones.

4.6 Continuous Evaluation (CI/CD)

For production GraphRAG systems, evaluation should run automatically on every code change — just like unit tests. This catches regressions before they reach users. The pipeline below integrates RAGAS into a GitHub Actions workflow that fails the build if quality drops.

RAGAS CI/CD Quality Gate

```

# graphrag_eval.py - RAGAS CI/CD quality gate
import sys, json
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall
)
from datasets import Dataset
import pandas as pd

# Quality thresholds - adjust per domain requirements
THRESHOLDS = {
    'faithfulness': 0.85,
    'answer_relevancy': 0.80,
    'context_precision': 0.75,
    'context_recall': 0.70,
}

def run_evaluation(rag_function, testset_csv: str) -> dict:
    df = pd.read_csv(testset_csv)
    answers, contexts = [], []
    for _, row in df.iterrows():
        result = rag_function(row['question'])
        answers.append(result['answer'])
        contexts.append(result['contexts'])
    dataset = Dataset.from_dict({
        'question': df['question'].tolist(),

```



```

'ground_truth': df['ground_truth'].tolist(),
'answer': answers,
'contexts': contexts,
})
results = evaluate(
dataset=dataset,
metrics=[faithfulness, answer_relevancy,
context_precision, context_recall]
)
scores = results.to_pandas().mean().to_dict()
passed = all(
scores[m] >= THRESHOLDS[m] for m in THRESHOLDS
)
return {'scores': scores, 'passed': passed}

if __name__ == '__main__':
from my_graphrag import hotel_rag_function
result = run_evaluation(
hotel_rag_function, 'hotel_kg_testset.csv'
)
print(json.dumps(result, indent=2))
sys.exit(0 if result["passed"] else 1)

```

GitHub Actions RAGAS CI/CD Pipeline

```

# .github/workflows/graphrag_eval.yml
name: GraphRAG Evaluation
on: [push, pull_request]
jobs:
  evaluate:
    runs-on: ubuntu-latest
    services:
      neo4j:
        image: neo4j:5.18-community
        env:
          NE04J_AUTH: neo4j/mas_memory_2024
          NE04J_PLUGINS: '["apoc"]'
        ports: ['7687:7687', '7474:7474']
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: pip install ragas langchain-anthropic neo4j
      - name: Load test data into Neo4j
        run: python scripts/load_test_data.py
      - name: Run RAGAS evaluation
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: python graphrag_eval.py
      - name: Upload RAGAS results artifact
        uses: actions/upload-artifact@v4
        with:
          name: ragas-results
          path: eval_results.json

```

Key Insight:

A RAGAS CI/CD gate that fails the build when faithfulness drops below 0.85 prevents prompt regressions from reaching production. Common causes of regression: changing LLM model version, modifying the system prompt, adding new node labels (schema change confuses Text2Cypher), or upgrading the embedding model (shifts vector space and breaks similarity thresholds).

4.7 Production Monitoring for GraphRAG

Evaluation during development is a prerequisite; monitoring in production is mandatory. Unlike traditional software, GraphRAG systems can degrade silently as the knowledge graph evolves, query patterns change, or the LLM provider updates their model without notice.

Metric	Type	Alert Threshold	How to Collect
Cypher success rate	Operational	Alert if below 95% of Cypher queries succeed	Wrap graph.query() in try/except; increment counter on failure
Query latency p99	Operational	Alert if p99 exceeds 3 seconds for simple queries	Time every graph.query() call; export to Prometheus or Datadog
Faithfulness (sampled)	Quality	Alert if rolling average drops below 0.85	Sample 5 percent of queries; run RAGAS faithfulness asynchronously
Empty result rate	Quality	Alert if more than 10 percent of queries return empty results	Check len(result) == 0 after every Cypher execution
Token usage per query	Cost	Alert if average exceeds 3000 tokens per query	Track LLM callback token counts; store in time-series DB
Graph node count	Data	Alert if growth rate exceeds 2x daily baseline	Run daily: MATCH (n) RETURN count(n); store in metrics DB

Production Monitoring Wrapper

```
# Production monitoring wrapper
import time
from functools import wraps
from collections import defaultdict

metrics = defaultdict(list)

def monitored_graph_query(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        try:
            result = func(*args, **kwargs)
            latency = time.time() - start
            metrics['query_latency'].append(latency)
            metrics['success_count'].append(1)
            if not result:
                metrics['empty_results'].append(1)
            return result
        except Exception as e:
            metrics['error_count'].append(1)
            metrics['errors'].append(str(e)[:100])
            raise
        return wrapper

@monitored_graph_query
def query_graph(cypher: str, params: dict = None):
    with driver.session() as s:
        return list(s.run(cypher, **(params or {})))

def get_metrics_summary() -> dict:
    n = len(metrics['success_count']) + len(metrics['error_count'])
    if n == 0:
        return {}
    lat = metrics['query_latency']
    return {
        'total_queries': n,
        'success_rate': len(metrics['success_count']) / n,
```

```
'empty_rate': len(metrics['empty_results']) / n,  
'avg_latency_ms': sum(lat)/len(lat)*1000 if lat else 0,  
'p99_latency_ms': sorted(lat)[int(len(lat)*0.99)]*1000  
if lat else 0  
}
```

Key Insight:

Track the empty result rate carefully. An empty Cypher result is often not an error — the LLM generated valid Cypher that matched no data. But if the empty rate spikes from 5 percent to 20 percent, it signals that either your schema changed, a bulk delete removed expected data, or the LLM is generating queries for node labels that no longer exist.

PART 5: Real-World Use Cases and Reference Implementations

Hotel IoT · Supply Chain · Healthcare · Financial Networks

This section walks through complete GraphRAG implementation blueprints for four industry domains. Each blueprint includes the domain model, critical Cypher queries, agent integration pattern, and common pitfalls. Use these as starting points for your Week 9 project.

5.1 Hotel IoT Knowledge Graph

The Hotel IoT domain is the primary example used throughout this course. A hotel has rooms, each with multiple IoT devices (HVAC, lighting, sensors). Devices generate continuous readings and trigger alerts. Staff resolve alerts via maintenance jobs. The knowledge graph connects all of this into a queryable operational picture that agents can reason over in real time.

Key business questions the KG must answer: Which rooms are currently too hot? Which devices have triggered the most alerts this week? Which staff member has the fastest resolution time? Which floors consume the most energy? Are there patterns between room type and device failure rate?

Query Pattern	Cypher Template	Agent Use Case
Rooms above temperature threshold	MATCH (r:Room)-[:HAS_READING]->(s:SensorReading) WHERE s.temp_c > \$threshold AND s.ts > datetime()-duration({hours:1}) RETURN r.id, max(s.temp_c)	Alert dispatching agent finds rooms needing immediate attention
Device alert frequency ranking	MATCH (d:Device)-[:TRIGGERED]->(a:Alert) WHERE a.ts > datetime()-duration({days:7}) RETURN d.id, count(a) AS alerts ORDER BY alerts DESC LIMIT 10	Maintenance planning agent prioritises device replacement
Staff resolution time	MATCH (s:Staff)-[:PERFORMED]->(j:MaintenanceJob) WHERE j.completed_at IS NOT NULL RETURN s.name, avg(duration.between(j.started_at,j.completed_at).minutes) AS avg_min	Workforce analytics agent measures technician performance
Floor energy consumption	MATCH (r:Room)-[:HAS_READING]->(s:SensorReading) WHERE s.ts > datetime()-duration({hours:24}) RETURN r.floor, sum(s.energy_kwh) AS total_kwh ORDER BY total_kwh DESC	Energy optimisation agent identifies high-consumption floors

Hotel IoT ReAct Agent — Full Implementation

```
# Complete hotel IoT agent implementation
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool
from langchain_anthropic import ChatAnthropic
from neo4j import GraphDatabase
from datetime import datetime

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

@tool
def get_hot_rooms(threshold_c: float = 28.0) -> str:
    """Find all rooms currently above the temperature threshold.
```

```

Args: threshold_c: temperature in Celsius (default 28.0)
"""

with driver.session() as s:
    results = s.run("""
MATCH (r:Room)-[:HAS_READING]->(sr:SensorReading)
WHERE sr.temp_c > $t
AND sr.ts > datetime() - duration({minutes: 30})
RETURN r.id AS room, r.floor AS floor,
max(sr.temp_c) AS max_temp
ORDER BY max_temp DESC
""", t=threshold_c)
    return str([dict(r) for r in results])

@tool
def get_unresolved_alerts(severity: str = "HIGH") -> str:
    """Get all unresolved alerts of a given severity level.
    Args: severity: one of HIGH, MEDIUM, LOW
    """
    with driver.session() as s:
        results = s.run("""
MATCH (d:Device)-[:TRIGGERED]->(a:Alert)
MATCH (d)-[:HAS_DEVICE]-(r:Room)
WHERE a.severity = $sev AND a.resolved = false
RETURN r.id AS room, d.id AS device,
a.type AS alert_type,
a.ts AS triggered_at
ORDER BY a.ts DESC LIMIT 20
""", sev=severity)
        return str([dict(r) for r in results])

@tool
def assign_maintenance(room_id: str,
    staff_id: str,
    job_type: str) -> str:
    """Create and assign a maintenance job to a staff member.
    Args:
    room_id: Room node id
    staff_id: Staff node id
    job_type: type of maintenance (e.g. HVAC_REPAIR)
    """
    with driver.session() as s:
        s.run("""
MATCH (r:Room {id: $rid}),
(st:Staff {id: $sid})
CREATE (j:MaintenanceJob {
id: randomUUID(),
type: $jtype,
started_at: datetime(),
status: "ASSIGNED"
})
CREATE (st)-[:PERFORMS]->(j)
CREATE (j)-[:FOR_ROOM]->(r)
""", rid=room_id, sid=staff_id, jtype=job_type)
        return f"Maintenance job created: {job_type} in {room_id}"

hotel_agent = create_react_agent(
    model=ChatAnthropic(model="claude-3-5-sonnet-20241022"),
    tools=[get_hot_rooms, get_unresolved_alerts, assign_maintenance],
    prompt="You are a hotel operations AI. Use the knowledge graph "
    "to monitor room conditions and coordinate maintenance. "
    "Always check for hot rooms and unresolved HIGH alerts first."
)

```

Key Insight: Tool naming matters enormously for agent performance. `get_hot_rooms` is clearer than `query_graph` with a Cypher parameter — the agent can reason about what the tool does from its name alone. Use domain-specific tool names and put the Cypher inside the tool function, hidden from the agent.

5.2 Supply Chain Knowledge Graph

Supply chains are inherently graph-structured: products are sourced from suppliers, processed at facilities, shipped via routes, and delivered to customers. Disruptions propagate through the network. A knowledge graph captures this topology and enables multi-hop impact analysis — "if Supplier X fails, which products are affected and which customers will be delayed?"

The graph model uses: Supplier nodes with country and reliability properties; Component nodes with lead time and safety stock; Product nodes; Facility nodes for manufacturing; Customer nodes; relationships: `SUPPLIES`, `REQUIRES`, `ASSEMBLES_AT`, `SHIPS_TO`, `ORDERS`.

Supply Chain Impact Analysis — Cypher

```
-- Supply chain impact analysis: supplier disruption
-- Question: if Supplier X stops delivering, what is affected?
MATCH (s:Supplier {id: $supplier_id})
-[:SUPPLIES]->(c:Component)
-[:REQUIRED_BY]->(p:Product)
-[:ORDERED_BY]->(customer:Customer)
RETURN s.name AS disrupted_supplier,
collect(DISTINCT c.name) AS affected_components,
collect(DISTINCT p.name) AS affected_products,
count(DISTINCT customer) AS customers_at_risk

-- Find alternative suppliers for a component
MATCH (c:Component {id: $component_id})
<-[:SUPPLIES]-(alt:Supplier)
WHERE alt.id <> $current_supplier
AND alt.reliability_score > 0.8
AND alt.lead_time_days <= c.max_lead_time
RETURN alt.name, alt.country,
alt.reliability_score,
alt.lead_time_days
ORDER BY alt.reliability_score DESC

-- Identify single-supplier dependencies (risk nodes)
MATCH (c:Component)
WITH c, size([(c)<-[:SUPPLIES]-( ) | 1]) AS supplier_count
WHERE supplier_count = 1
MATCH (c)<-[:SUPPLIES]-(s:Supplier)
RETURN c.name AS component,
s.name AS sole_supplier,
s.country AS risk_country
```

Graph algorithms for supply chain: Use GDS Betweenness Centrality to find the most critical nodes in the supply network (removing them would disconnect the graph). Use GDS Weakly Connected Components to find isolated supply chains that are not connected to the main network. Use GDS PageRank on the `SUPPLIES` relationship to find the most influential suppliers.

GDS Betweenness Centrality for Supply Risk

```
-- GDS Betweenness Centrality: find critical supply chain nodes
CALL gds.graph.project(
'supplyGraph',
['Supplier', 'Component', 'Product', 'Customer'],
['SUPPLIES', 'REQUIRED_BY', 'ORDERED_BY']
)

CALL gds.betweenness.stream('supplyGraph')
YIELD nodeId, score
```

```

RETURN gds.util.asNode(nodeId).name AS node,
labels(gds.util.asNode(nodeId))[0] AS type,
score AS centrality
ORDER BY centrality DESC LIMIT 10

-- High centrality = critical path node
-- Remove it and the supply chain splits

```

Supply Chain Graph ROI

A supply chain knowledge graph enables questions that are impossible with spreadsheets or traditional ERP: "Find all products that share a common single-source component from a country with active trade sanctions." This multi-hop query over supplier-component-product-country relationships takes milliseconds in Neo4j vs. hours of manual cross-referencing.

5.3 Healthcare Knowledge Graph

Healthcare knowledge graphs connect patients, diagnoses, medications, procedures, providers, and clinical guidelines. The graph structure enables clinical decision support: "Does this new medication interact with any of the patient's current prescriptions?" or "What similar patients had good outcomes with Treatment X?"

Critical compliance note: Healthcare graphs containing real patient data must comply with HIPAA (US), GDPR (EU), and PDPA (Thailand). Use synthetic or de-identified data for development. In production, all node properties containing PHI must be encrypted at rest and in transit. Neo4j Enterprise provides native property-level encryption.

Healthcare KG — Clinical Decision Support Queries

```

-- Drug interaction detection
MATCH (patient:Patient {id: $patient_id})
-[:PRESCRIBED]->(existing:Medication)
-[:INTERACTS_WITH]->(new_drug:Medication {id: $new_drug_id})
RETURN existing.name AS existing_drug,
new_drug.name AS proposed_drug,
type(r) AS interaction_type,
r.severity AS severity

-- Find similar patients (cohort analysis)
MATCH (p:Patient {id: $patient_id})
-[:HAS_DIAGNOSIS]->(d:Diagnosis)
<-[:HAS_DIAGNOSIS]-(similar:Patient)
WHERE similar.age_group = p.age_group
AND similar.id <> p.id
WITH similar, count(d) AS shared_diagnoses
WHERE shared_diagnoses >= 2
MATCH (similar)-[:RECEIVED]->(t:Treatment)
WHERE (similar)-[:RESPONDED_POSITIVELY]->(t)
RETURN t.name AS treatment,
count(similar) AS patients_responded,
avg(t.response_rate) AS avg_response
ORDER BY patients_responded DESC LIMIT 5

```

Graph Algorithm	Clinical Application	GDS Function
Community Detection	Identify patient cohorts with similar diagnoses and outcomes	<code>gds.louvain.stream('clinicalGraph')</code>
Link Prediction	Predict likely diagnoses from patient symptom graph	<code>gds.linkPrediction.pipeline.predict.stream()</code>
Node Similarity	Find similar patients for cohort studies and trial matching	<code>gds.nodeSimilarity.stream('patientGraph')</code>

Graph Algorithm	Clinical Application	GDS Function
PageRank on Treatments	Rank treatments by clinical guideline centrality	<code>gds.pageRank.stream('guidelineGraph')</code>
Shortest Path	Find care pathway between admission and discharge	<code>gds.shortestPath.dijkstra.stream('careGraph', ...)</code>

5.4 Financial Network Knowledge Graph

Financial knowledge graphs model entities (accounts, companies, people, transactions) and their relationships (TRANSFERS_TO, OWNED_BY, WORKS_FOR, GUARANTEES). Graph algorithms on this network enable fraud detection, AML (anti-money laundering), and counterparty risk analysis that pattern-matching rules cannot catch.

The power of graph-based fraud detection: a fraudulent transaction network creates characteristic topological patterns (rings, stars, cascades) that are invisible in tabular data but immediately visible in a graph. GDS community detection can identify fraud rings in seconds that would take days of manual investigation.

Financial Network — Fraud Detection Queries

```
-- Fraud ring detection: circular money flows
MATCH path = (a:Account)-[:TRANSFERS_TO*2..5]->(a)
WHERE ALL(r IN relationships(path)
WHERE r.amount > 10000
AND r.ts > datetime() - duration({hours: 24}))
RETURN path,
length(path) AS ring_size,
reduce(total=0, r IN relationships(path) | total + r.amount)
AS total_circulated
ORDER BY total_circulated DESC

-- Entity relationship network for counterparty risk
MATCH (company:Company {id: $company_id})
-[:OWNS|CONTROLLED_BY|AFFILIATED_WITH*1..4]->(related)
RETURN labels(related)[0] AS entity_type,
related.name AS name,
related.risk_score AS risk,
length(shortestPath(
(company)-[:OWNS|CONTROLLED_BY|AFFILIATED_WITH*]->(related)
)) AS degrees_of_separation
ORDER BY risk DESC LIMIT 20
```

GDS Louvain for Fraud Ring Detection

```
-- GDS: Detect suspicious transaction communities
CALL gds.graph.project(
'transactionGraph',
['Account', 'Company', 'Person'],
{TRANSFERS_TO: {properties: ['amount', 'ts']}}
)

-- Louvain to find tight transaction clusters
CALL gds.louvain.stream('transactionGraph')
YIELD nodeId, communityId
WITH communityId,
collect(gds.util.asNode(nodeId)) AS members
WHERE size(members) BETWEEN 3 AND 15 -- suspicious ring size
RETURN communityId,
size(members) AS ring_size,
[m IN members | m.id] AS account_ids
ORDER BY ring_size DESC
```


Key Insight:

Financial graphs require strict access control — not every agent should see all relationships. Use Neo4j role-based access control to restrict subgraph visibility: compliance agents see all; customer-facing agents see only their own account subgraph. Graph security is a first-class concern in regulated industries.

PART 6: Reference Tables and Quick Guides

Cypher Cheat Sheet · Framework Comparison · Tool Selection Guide

6.1 Cypher Quick Reference

Use this reference table while writing Cypher queries. All patterns are compatible with Neo4j 5.x and the OpenCypher standard.

Category	Syntax	Example
Node pattern	(n:Label {prop: value})	(r:Room {id: "R301"})
Relationship pattern	-[:TYPE {prop: val}]->	-[:KNOWS {since: 2020}]->
Variable-length path	-[:TYPE*min..max]->	-[:SUPPLIES*1..5]->
Any direction	-[:TYPE]-	(a)-[:KNOWS]-(b)
Bidirectional	<-[:TYPE]->	(a)<-[:REPORTS_TO]->(b)
Named relationship	[r:TYPE]	(a)-[r:KNOWS]->(b) WHERE r.since>2020
CREATE	CREATE (n:Label {props})	CREATE (p:Person {name:"Alice"})
MERGE	MERGE (n:Label {key: val})	MERGE (p:Person {name:\$name})
MATCH + WHERE	MATCH (n) WHERE n.x > \$val	MATCH (r:Room) WHERE r.floor=3
OPTIONAL MATCH	OPTIONAL MATCH (n)-[:R]->(m)	Returns null if m not found (LEFT JOIN)
WITH	WITH n, count(r) AS cnt	Passes variables forward; enables HAVING-style filter
UNWIND	UNWIND list AS item	UNWIND [1,2,3] AS n RETURN n*2
COLLECT	collect(n.prop)	Returns list of values; groups like SQL GROUP BY
SIZE of list	size(list)	size([(n)-[:R]->() 1]) AS degree
CASE	CASE WHEN x THEN y ELSE z END	CASE WHEN r.temp>28 THEN "HOT" ELSE "OK" END
FOREACH	FOREACH (x IN list SET x.p=v)	FOREACH (n IN nodes(path) SET n.visited=true)
CALL {}	CALL { WITH n MATCH ... RETURN }	Correlated subquery — use WITH to pass variables
EXISTS {}	WHERE EXISTS { MATCH ... }	WHERE EXISTS { (p)-[:MANAGES]->(:Team) }
Pattern comprehension	[(n)-[:R]->(m) m.prop]	Returns list of matched values inline
shortestPath	shortestPath((a)-[*]-(b))	BFS shortest path — no weight
duration arithmetic	datetime()-duration({days:7})	Returns datetime 7 days ago
String functions	STARTS WITH, ENDS WITH, CONTAINS	WHERE n.name STARTS WITH "AI"
toInteger / toFloat	toInteger(str), toFloat(str)	Type coercion for CSV imports
randomUUID()	randomUUID()	Generate unique ID for new nodes
DETACH DELETE	DETACH DELETE n	Delete node and all its relationships

Category	Syntax	Example
REMOVE label	REMOVE n:LabelName	Remove a label from a node
SET label	SET n:NewLabel	Add a label to a node dynamically

6.2 APOC Procedures Quick Reference

Function / Procedure	Purpose	Example Call
apoc.load.json(url)	Load JSON from URL or file path	CALL apoc.load.json('file:///data.json') YIELD value
apoc.load.csv(url, config)	Load CSV with options (delimiter, header)	CALL apoc.load.csv('file:///rooms.csv') YIELD map
apoc.periodic.iterate(query, action, config)	Batch processing to avoid OOM	CALL apoc.periodic.iterate('MATCH (n:Room) RETURN n', 'SET n.processed=true', {batchSize:100})
apoc.create.node(labels, props)	Create node with dynamic label list	CALL apoc.create.node(['Room', 'Suite'], {id:'R1'}) YIELD node
apoc.create.relationship(from, type, props, to)	Create relationship with dynamic type	CALL apoc.create.relationship(a, 'KNOWS', {since:2020}, b) YIELD rel
apoc.refactor.mergeNodes(nodes, config)	Merge duplicate nodes	CALL apoc.refactor.mergeNodes([n1, n2], {properties:'combine'})
apoc.util.sha256(list)	Generate SHA-256 hash for deduplication	RETURN apoc.util.sha256(['alice@email.com'])
apoc.map.merge(map1, map2)	Merge two maps into one	RETURN apoc.map.merge({a:1}, {b:2}) AS merged
apoc.date.parse(str, unit, fmt)	Parse date string to epoch	RETURN apoc.date.parse('2024-01-15', 'ms', 'y yy-MM-dd')
apoc.coll.toSet(list)	Remove duplicates from a list	RETURN apoc.coll.toSet(['a', 'b', 'a']) AS unique
apoc.text.split(str, regex)	Split string by regex pattern	RETURN apoc.text.split('a,b,c', ',') AS parts

6.3 GDS Algorithms Quick Reference

Algorithm	Category	Use Case	Write Property
gds.pageRank	Centrality	Find most influential nodes	importance_score
gds.betweenness	Centrality	Find bridge nodes in network	betweenness_score
gds.degree	Centrality	Count connections per node	degree
gds.louvain	Community	Detect clusters of tightly-connected nodes	community_id
gds.labelPropagation	Community	Semi-supervised community detection	community_id
gds.wcc (Weakly Connected Components)	Community	Find isolated subgraphs	component_id
gds.fastRP	Embedding	Graph-structural node embeddings (fast)	embedding
gds.node2vec	Embedding	Random-walk structural embeddings	n2v_embedding

Algorithm	Category	Use Case	Write Property
gds.graphSage	Embedding	Inductive embeddings using node features	sage_embedding
gds.shortestPath.dijkstra	Path	Weighted shortest path between two nodes	path (stream)
gds.allShortestPaths.delta	Path	All shortest paths (parallel Dijkstra)	path (stream)
gds.nodeSimilarity	Similarity	Find similar nodes by shared neighbours	similarity_score
gds.knn	Similarity	K-nearest neighbours by node embedding	score
gds.linkPrediction.pipeline	ML	Predict missing edges in graph	probability

6.4 Development Environment Setup

Use this setup guide to prepare your development environment for all code examples in this course. All examples were tested with the versions listed below.

Development Environment Setup

1. Start Neo4j with Docker (recommended for development)

```
docker run -d \
--name neo4j-kg \
-p 7474:7474 -p 7687:7687 \
-e NEO4J_AUTH=neo4j/mas_memory_2024 \
-e NEO4J_PLUGINS='["apoc","graph-data-science"]' \
-v $HOME/neo4j_data:/data \
neo4j:5.18-community
```

2. Install Python dependencies

```
pip install \
neo4j==5.19.0 \
langchain==0.2.16 \
langchain-community==0.2.16 \
langchain-anthropic==0.1.23 \
langchain-experimental==0.0.65 \
langgraph==0.2.27 \
crewai==0.67.0 \
google-adk==0.5.0 \
fastmcp==0.4.0 \
llama-index==0.11.0 \
llama-index-graph-stores-neo4j==0.3.0 \
ragas==0.1.19 \
langchain-openai==0.1.23 \
datasets==2.21.0
```

3. Verify Neo4j connection

```
python -c "
from neo4j import GraphDatabase
driver = GraphDatabase.driver('bolt://localhost:7687',
auth=('neo4j','mas_memory_2024'))
with driver.session() as s:
print(list(s.run('RETURN 1 AS n'))[0]['n'])
" # Should print: 1
```

4. Verify APOC and GDS are installed

Open <http://localhost:7474> in browser

Run in query box:

```
RETURN apoc.version() AS apoc, gds.version() AS gds
```

Tool / Library	Version Tested	Install Command
Neo4j	5.18 Community	<code>docker pull neo4j:5.18-community</code>
APOC	5.18.0	Set NEO4J_PLUGINS env var (auto-install)
GDS (Graph Data Science)	2.6.0	Set NEO4J_PLUGINS env var (auto-install)
Python neo4j driver	5.19.0	<code>pip install neo4j==5.19.0</code>
LangChain	0.2.16	<code>pip install langchain langchain-community</code>
LangGraph	0.2.27	<code>pip install langgraph</code>
CrewAI	0.67.0	<code>pip install crewai</code>
Google ADK	0.5.0	<code>pip install google-adk</code>
FastMCP	0.4.0	<code>pip install fastmcp</code>
LlamaIndex	0.11.0	<code>pip install llama-index llama-index-graph-stores-neo4j</code>
RAGAS	0.1.19	<code>pip install ragas</code>
OpenAI SDK (embeddings)	1.43.0	<code>pip install openai langchain-openai</code>
Anthropic SDK	0.34.0	<code>pip install anthropic langchain-anthropic</code>

6.5 Tool and Framework Selection Guide

Use this decision tree to choose the right combination of tools for your GraphRAG project. Start with your primary deployment target and follow the branches.

Framework Selection Decision Tree

GRAPHRAG TOOL SELECTION DECISION TREE
=====

Q1: What is your primary deployment target?

- > Google Cloud / Vertex AI Google ADK + Neo4j
- > AWS / Azure / Self-hosted LangGraph or CrewAI
- > Claude Desktop (personal tool) FastMCP + Neo4j
- > Jupyter / research / prototype LangChain or LlamaIndex

Q2: How many agents do you need?

- > Single agent with tools LangGraph (create_react_agent)
- > Multiple agents with defined roles CrewAI
- > Dynamic agent graph (custom flow) LangGraph (StateGraph)

Q3: What is your primary retrieval pattern?

- > NL-to-Cypher (structured data) GraphCypherQAChain (LangChain)
- > Semantic search + graph expand Neo4jVector + Cypher (LangChain)
- > Document triple extraction LlamaIndex KnowledgeGraphIndex
- > LLM entity extraction from text LLMGraphTransformer (LangChain)

Q4: What is your LLM preference?

- > Claude (Anthropic) LangChain + ChatAnthropic
- > Gemini (Google) Google ADK + Gemini
- > GPT-4 / GPT-4o (OpenAI) LangChain + ChatOpenAI
- > Local / open source models LangChain + Ollama integration

Key Insight:

For Week 9 project work: start with LangChain GraphCypherQAChain for the simplest path from question to graph answer. Add LangGraph ReAct agent tools when you need multi-step reasoning. Use CrewAI when you want multiple specialized agents. Save Google ADK and FastMCP for production deployments — they add operational overhead not needed during development.

PART 7: Workshop Labs and Week 9 Project Guide

Hands-On Labs · Project Rubric · Submission Checklist

This section provides structured lab exercises that reinforce each major concept in the course, followed by the Week 9 project specification and evaluation rubric. Complete all labs before starting the project.

LAB 1 — Setting Up the Hotel IoT Knowledge Graph

Objective: Create a complete hotel IoT knowledge graph from scratch using Cypher and Python. Duration: 45 minutes.

Prerequisites: Neo4j 5.x running (Docker), neo4j Python driver installed, APOC plugin enabled.

Steps:

Lab 1: Hotel KG Setup Script

```
# Step 1: Create constraints (run in Neo4j Browser)
CREATE CONSTRAINT room_unique FOR (r:Room) REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT device_unique FOR (d:Device) REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT staff_unique FOR (s:Staff) REQUIRE s.id IS UNIQUE;

# Step 2: Create room nodes (Python)
from neo4j import GraphDatabase
driver = GraphDatabase.driver('bolt://localhost:7687',
auth=('neo4j','mas_memory_2024'))

rooms = [
{'id':'R101','floor':1,'type':'Standard','capacity':2,'temp_c':24},
{'id':'R201','floor':2,'type':'Deluxe','capacity':2,'temp_c':26},
{'id':'R301','floor':3,'type':'Suite','capacity':4,'temp_c':29},
{'id':'R302','floor':3,'type':'Suite','capacity':4,'temp_c':22},
{'id':'R401','floor':4,'type':'Standard','capacity':2,'temp_c':25},
]

with driver.session() as s:
for room in rooms:
s.run("""
MERGE (r:Room {id: $id})
SET r.floor=$floor, r.type=$type,
r.capacity=$capacity
""", **room)

# Step 3: Create device and staff nodes
devices = [
{'id':'HVAC-R101','type':'HVAC','room_id':'R101'},
{'id':'HVAC-R301','type':'HVAC','room_id':'R301'},
{'id':'SENSOR-R301','type':'TempSensor','room_id':'R301'},
]

with driver.session() as s:
for dev in devices:
s.run("""
MERGE (d:Device {id: $id})
SET d.type = $type
WITH d
MATCH (r:Room {id: $room_id})
MERGE (r)-[:HAS_DEVICE]->(d)
""", **dev)

# Step 4: Create an alert
```

```

with driver.session() as s:
    s.run("""
MATCH (d:Device {id: 'HVAC-R301'})
CREATE (a:Alert {
id: 'ALT-001',
type: 'HIGH_TEMP',
severity: 'HIGH',
ts: datetime(),
resolved: false
})
CREATE (d)-[:TRIGGERED]->(a)
""")

```

Verification query: Run in Neo4j Browser to confirm setup.

Lab 1: Verification Query

```

MATCH (r:Room)-[:HAS_DEVICE]->(d:Device)
OPTIONAL MATCH (d)-[:TRIGGERED]->(a:Alert)
RETURN r.id, r.floor, r.type,
count(d) AS devices,
count(a) AS alerts
ORDER BY r.floor

```

Expected result: 5 rooms, each with at least 1 device. Room R301 should show 1 alert.

LAB 2 — LLMGraphTransformer Entity Extraction

Objective: Extract entities from a maintenance report using LLMGraphTransformer and add them to the hotel KG.

Duration: 30 minutes.

Prerequisites: Completed Lab 1. Anthropic API key set.

Lab 2: LLMGraphTransformer Extraction

```

# Lab 2: LLMGraphTransformer extraction
from langchain_experimental.graph_transformers import LLMGraphTransformer
from langchain_community.graphs import Neo4jGraph
from langchain_anthropic import ChatAnthropic
from langchain_core.documents import Document

# Sample maintenance report (use your own for the project)
REPORT = """
Maintenance Report - 2024-01-15
Technician: Somchai Rattana (Tech ID: T-045)

Issue: HVAC unit HVAC-R301 in Room 301 triggered a HIGH TEMP alert
at 14:23. Room temperature reached 29.5 degrees Celsius.

Root cause: Clogged air filter (filter model: Daikin AF-200).
The filter had not been replaced for 180 days, exceeding the
90-day recommended replacement interval.

Resolution: Replaced air filter. HVAC recalibrated to 22C setpoint.
Room temperature returned to normal by 15:45.

Recommendation: Schedule quarterly filter replacement for all
HVAC units on Floor 3. Supplier: Bangkok HVAC Parts Co., Ltd.
"""

graph = Neo4jGraph(
    url='bolt://localhost:7687',
    username='neo4j', password='mas_memory_2024'
)

```



```

llm = ChatAnthropic(model="claude-3-5-sonnet-20241022")

transformer = LLMGraphTransformer(
    llm=llm,
    allowed_nodes=['Room', 'Device', 'Staff', 'Alert',
                  'MaintenanceJob', 'Supplier', 'Component'],
    allowed_relationships=[
        'HAS_DEVICE', 'TRIGGERED', 'ASSIGNED_TO',
        'RESOLVED_BY', 'SUPPLIES', 'USED_IN'
    ]
)

docs = [Document(page_content=REPORT)]
graph_docs = transformer.convert_to_graph_documents(docs)
graph.add_graph_documents(graph_docs, include_source=True)

print(f"Nodes: {[n.id for n in graph_docs[0].nodes]}")
print(f"Rel: {[r.source.id, r.type, r.target.id] for r in graph_docs[0].relationships}")

```

Discussion questions: (1) Which entities were extracted correctly? (2) Were any entity types hallucinated? (3) How would you add an "allowed_nodes" constraint to improve precision?

LAB 3 — Build a GraphRAG Agent with LangGraph

Objective: Build a LangGraph ReAct agent that can query the hotel KG and answer operations questions. Duration: 60 minutes.

Deliverable: Agent that correctly answers all 5 test questions.

Lab 3: Hotel Operations Agent

```

# Lab 3: Build and test a hotel operations agent
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool
from langchain_anthropic import ChatAnthropic
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'mas_memory_2024')
)

def run_cypher(cypher: str, params: dict = None) -> list:
    with driver.session() as s:
        return [dict(r) for r in s.run(cypher, **(params or {}))]

@tool
def get_rooms_by_floor(floor: int) -> str:
    """Get all rooms on a specific floor with their current status."""
    results = run_cypher(
        "MATCH (r:Room {floor: $f}) RETURN r.id, r.type, r.capacity",
        {"f": floor}
    )
    return str(results)

@tool
def get_active_alerts(severity: str = "HIGH") -> str:
    """Get all currently active alerts by severity (HIGH/MEDIUM/LOW)."""
    results = run_cypher(
        "MATCH (d:Device)-[:TRIGGERED]->(a:Alert {severity:$s, resolved:false}) "
        "MATCH (d)-[:HAS_DEVICE]-(r:Room) "
        "RETURN r.id, a.type, a.ts",
    )

```

```

{"s": severity}
)
return str(results)

@tool
def count_devices_by_type(device_type: str) -> str:
    """Count how many devices of a given type exist in the hotel."""
    results = run_cypher(
        "MATCH (d:Device {type: $t}) RETURN count(d) AS count",
        {"t": device_type}
    )
    return str(results)

agent = create_react_agent(
    model=ChatAnthropic(model="claude-3-5-sonnet-20241022"),
    tools=[get_rooms_by_floor, get_active_alerts, count_devices_by_type],
    prompt="You are a hotel operations assistant with access to the "
    "hotel knowledge graph. Answer questions accurately using "
    "the graph tools. Be concise."
)

# Test questions (your agent should answer all of these)
test_questions = [
    "What rooms are on floor 3?",
    "Are there any active HIGH severity alerts?",
    "How many HVAC devices does the hotel have?",
    "What type is Room R301?",
    "Which room triggered the most recent alert?",
]

for q in test_questions:
    print(f"\nQ: {q}")
    result = agent.invoke({"messages": [{"role": "user", "content": q}]})
    print(f'A: {result["messages"][-1].content}')

```

LAB 4 — Evaluate Your Agent with RAGAS

Objective: Run a RAGAS evaluation on your Lab 3 agent and identify one area for improvement. Duration: 30 minutes.

Lab 4: RAGAS Evaluation

```

# Lab 4: RAGAS evaluation of hotel agent
# Note: requires ANTHROPIC_API_KEY environment variable
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from ragas.llms import LangchainLLMWrapper
from langchain_anthropic import ChatAnthropic
from datasets import Dataset

# Manually crafted test set for lab
test_data = {
    "question": [
        "What rooms are on floor 3?",
        "Are there any HIGH severity alerts?",
    ],
    "ground_truth": [
        "Rooms R301 and R302 are on floor 3.",
        "Yes, Room R301 has a HIGH severity HIGH_TEMP alert.",
    ],
    "contexts": [
        ["Room R301: floor=3, type=Suite", "Room R302: floor=3, type=Suite"],
        ["Alert ALT-001: type=HIGH_TEMP, severity=HIGH, room=R301"],
    ]
}

```

```

],
"answer": [
  "Rooms R301 and R302 are on floor 3.",
  "Yes, there is a HIGH severity HIGH_TEMP alert in Room R301.",
],
}

```

```
dataset = Dataset.from_dict(test_data)
```

```

eval_llm = LangchainLLMWrapper(
  ChatAnthropic(model="claude-3-5-sonnet-20241022")
)

```

```

results = evaluate(
  dataset=dataset,
  metrics=[faithfulness, answer_relevancy],
  llm=eval_llm
)

```

```
print(results.to_pandas())
```

```

# Lab reflection questions:
# 1. What is the faithfulness score? Is it above 0.85?
# 2. What would lower the faithfulness score?
# 3. How would you improve context recall for this agent?

```

WEEK 9 PROJECT SPECIFICATION

Apply everything from this course to build a complete GraphRAG system for your chosen domain. Your project must include: a Neo4j knowledge graph with at least 3 node types and 4 relationship types, at least one AI agent with KG tools, and a RAGAS evaluation with at least 10 test cases.

Component	Requirement	Points
Knowledge Graph Design	Domain model diagram + schema with constraints and indexes	15
Data Loading	At least one of: LOAD CSV, apoc.load.json, LLMGraphTransformer	15
Graph Enrichment	At least one GDS algorithm (PageRank, Louvain, or embeddings) applied	10
Agent Implementation	Working agent (any framework) with at least 3 KG query tools	25
GraphRAG Pipeline	End-to-end Q&A; working: user question -> KG query -> grounded answer	20
RAGAS Evaluation	Faithfulness ≥ 0.80 on 10+ test cases with iteration log	15
Code Quality	Clean Python, error handling, documented functions, README	10
Bonus: MCP Server	FastMCP server exposing the KG to Claude Desktop (+5 bonus points)	+5

Submission format: GitHub repository with README, requirements.txt, sample data, and a PDF report showing RAGAS scores and the iteration improvement log.

Key Insight: The most common project failure: building the graph without testing agent queries end-to-end first. Before adding any LLM, write 5 Cypher queries that answer your target business questions manually in Neo4j Browser. If those work, your GraphRAG implementation has a strong foundation.

PART 8: Glossary, Further Reading, and Course Resources

Key Terms · Academic References · Online Resources · Neo4j GraphAcademy

8.1 Knowledge Graph Glossary

A standardised vocabulary for knowledge graph and GraphRAG terminology used throughout this course and in the Neo4j and LangChain documentation.

Term	Definition
Knowledge Graph (KG)	A network of entities (nodes) connected by typed, directed relationships (edges), where both nodes and edges carry key-value properties. Enables multi-hop queries and semantic reasoning.
Node (Vertex)	A data entity in a graph. In Neo4j, nodes carry one or more labels (e.g., :Room, :Device) and a set of properties.
Edge (Relationship)	A directed, typed connection between two nodes. In Neo4j, relationships have a type (e.g., :KNOWS), direction, and optional properties.
Label	A categorical tag on a Neo4j node that groups it with similar nodes (e.g., :Person, :Room). Nodes can have multiple labels.
Property	A key-value attribute stored on a node or relationship. Values can be strings, numbers, booleans, lists, or temporal types.
Cypher	Neo4j's declarative graph query language. Uses ASCII-art notation: (n:Label)-[:REL_TYPE]->(m:Label). Now an open standard (GQL).
Index-free Adjacency	Neo4j's core storage innovation. Each node stores direct pointers to adjacent nodes, making single-hop traversal O(1) regardless of graph size.
APOC	Awesome Procedures On Cypher. The standard extension library for Neo4j with 450+ procedures for data loading, transformation, and graph operations.
GDS (Graph Data Science)	Neo4j plugin providing ML algorithms on graphs: centrality, community detection, link prediction, and node embeddings (PageRank, Louvain, FastRP).
RAG (Retrieval-Augmented Generation)	AI pattern where an LLM is given retrieved external context before generating an answer, reducing hallucination by grounding responses in retrieved facts.
GraphRAG	RAG where the retrieval step uses a knowledge graph. Combines vector similarity search with graph traversal for richer, more grounded context.
Vector Index	A data structure that enables approximate nearest-neighbour (ANN) search over high-dimensional embedding vectors. Neo4j uses HNSW natively.
Embedding	A dense vector representation of text, image, or graph structure. Similar items have similar (high cosine similarity) embeddings.
Chunk	A segment of a source document (fixed-size or semantic) stored as a :Chunk node with an embedding vector, enabling semantic search retrieval.
LLMGraphTransformer	LangChain component that uses an LLM to extract entities and relationships from unstructured text and convert them to Neo4j graph documents.
RAGAS	Retrieval-Augmented Generation Assessment. Evaluation framework that measures faithfulness, answer relevancy, context precision, and context recall.
Faithfulness	RAGAS metric: fraction of answer claims that are supported by retrieved context. High faithfulness (>0.90) means the answer is grounded, not hallucinated.

Term	Definition
Context Precision@K	RAGAS metric: are relevant chunks ranked before irrelevant ones in the top-K retrieved results? High score means retrieval ranking is accurate.
MCP (Model Context Protocol)	Anthropic's open standard for connecting AI models to external tools and data sources. FastMCP is the Python library for building MCP servers.
ReAct	Reason + Act. Agent loop where the LLM alternates between reasoning about what to do next and executing tool calls, iterating until the task is complete.
Index-free Adjacency	Core Neo4j storage design: each node stores physical pointers to adjacent nodes. Enables O(1) hop traversal independent of total graph size.
Triple Store	A database optimised for (subject, predicate, object) triples. LlamaIndex uses this pattern when extracting knowledge from documents.
Ontology	A formal specification of concepts and relationships in a domain. In KG design, ontologies define the allowed node labels, relationship types, and property schemas.
Bolt Protocol	Neo4j's binary wire protocol on port 7687. Used by all official Neo4j drivers (Python, JavaScript, Java, Go, .NET) for high-performance access.
BDI (Belief-Desire-Intention)	Woolridge's agent architecture. Beliefs map to KG nodes/edges; desires map to goal nodes; intentions map to active plan subgraphs.

8.2 Academic References

These papers and books provide the theoretical foundations for knowledge graphs, multi-agent systems, and retrieval-augmented generation.

Reference	Relevance to this Course
Woolridge, M. (2009). An Introduction to MultiAgent Systems (2nd ed.). Wiley.	Foundational MAS theory. BDI architecture maps directly to KG-backed agent memory (Ch.2).
Pan, J. Z. et al. (2023). Unifying Large Language Models and Knowledge Graphs. arXiv:2306.08302.	Survey of LLM+KG integration patterns. Directly motivates GraphRAG architecture.
Edge, D. et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.	Microsoft GraphRAG paper. Community detection for hierarchical graph summarisation.
Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.	Original RAGAS paper defining the four evaluation metrics used in this course.
Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.	ReAct agent loop paper. Foundation for LangGraph create_react_agent pattern.
Robinson, I., Webber, J., and Eifrem, E. (2015). Graph Databases. O'Reilly.	Comprehensive Neo4j reference. Node types, relationship types, Cypher query patterns.
Hamilton, W. L. et al. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017.	GraphSAGE paper. Foundation for GDS node embedding algorithms used in Section 2.5.
Mikolov, T. et al. (2013). Distributed Representations of Words and Phrases. NeurIPS 2013.	Word2Vec paper. Conceptual basis for Node2Vec and graph embedding algorithms.

8.3 Online Resources and Neo4j GraphAcademy

Resource	URL	What to Study
Neo4j GraphAcademy	graphacademy.neo4j.com	Free courses: Neo4j Fundamentals, Cypher Fundamentals, Knowledge Graph RAG (this course's reference)
Neo4j Docs	neo4j.com/docs	Cypher reference, APOC docs, GDS docs, vector index docs
LangChain Neo4j Docs	python.langchain.com/docs/integrations/graphs/neo4j_cypher	GraphCypherQAChain, Neo4jVector, LLMGraphTransformer integration guides
RAGAS Documentation	docs.ragas.io	Metric definitions, evaluation pipeline, TestsetGenerator API reference
FastMCP	github.com/jlowin/fastmcp	MCP server building — <code>@mcp.tool()</code> , <code>@mcp.resource()</code> , <code>mcp.run()</code>
LangGraph Documentation	langchain-ai.github.io/langgraph	<code>create_react_agent</code> , <code>StateGraph</code> , tool integration patterns
CrewAI Documentation	docs.crewai.com	Agent, Task, Crew, Process, BaseTool patterns and examples
Google ADK Documentation	google.github.io/adk-docs	Agent, FunctionTool, Runner, SessionService for Gemini agents
LlamaIndex Neo4j	docs.llamaindex.ai/en/stable/examples/index_structs/knowledge_graph	KnowledgeGraphIndex, Neo4jGraphStore, triple extraction
AltoTech Course Platform	agentic-coding-fitness.altotech.ai	Week 9 project submission, lab datasets, solution code, peer review

Neo4j GraphAcademy — Required Course

Before submitting the Week 9 project, complete the free Neo4j GraphAcademy course "Introduction to Vector Indexes and Unstructured Data" at graphacademy.neo4j.com. This course covers vector index creation, embedding storage, and KNN search — the technical foundation of all GraphRAG patterns in this course. Certificate of completion is required for full project marks. The reference course "Knowledge Graph and Retrieval-Augmented Generation (RAG)" at graphacademy.neo4j.com/knowledge-graph-rag is the primary reference for Weeks 8 and 9 of Agentic AI Coding Fitness.